SPARK / WAREHOUSE CHECK

Dokumentasi Kode Pengembang

Memahami sintaks, fungsi, dan alur program

Panduan membaca implementasi Python yang digunakan oleh aplikasi warehouse: dari perhitungan aging sampai keputusan, pemantauan QR, dan laporan.

`Acuan`	Kode proyek saat penyusunan, rilis Windows 1.0.1, SQLite schema 7.
`Pembaca`	Pengembang berikutnya serta pemilik proyek yang ingin memahami cara kerja kode.
`Bentuk`	Kode sumber bernomor baris + penjelasan sintaks + contoh input/output + alur dan error.

5 Oktober 2026

Rekomendasi menggunakan aturan dan data simulasi Site/Customer. Contoh kebutuhan bukan model AI yang telah dilatih.

Cara membaca panduan

Setiap bagian menunjukkan fungsi yang benar-benar ada pada proyek. Cuplikan dipilih untuk menjelaskan langkah penting; kode lengkap tetap berada pada nama file sumber yang disebutkan. Nomor di kiri adalah nomor baris file asli, sehingga pengembang dapat menemukan potongan itu dengan cepat.

Baris kode yang panjang dipatahkan secara visual dan diberi tanda > pada lanjutan. Nomor yang sama berarti tetap satu baris sumber. Pematahan ini membantu membaca PDF; bukan perubahan sintaks program.

Sintaks	Cara membacanya
`def / return`	Mendefinisikan fungsi dan mengembalikan hasil.
`self / class`	self menunjuk objek yang sedang menjalankan metode; class mengelompokkan state dan perilaku.
`dict / list / tuple`	Dictionary berisi key/value, list berupa urutan yang dapat diubah, tuple berupa urutan tetap.
`with / try / finally`	Mengelola resource/transaksi serta jalur gagal dan cleanup. with db tidak menutup koneksi.
`raise / except`	Melempar dan menangani exception; pemanggil UI biasanya mengubahnya menjadi pesan pengguna.
`f-string / :02d`	Memasukkan nilai ke teks dan memformat angka, misalnya 3 menjadi 03.
`? / :nama di SQL`	Placeholder parameter SQL; nilai diberikan terpisah dari perintah query.
`lambda / after`	Fungsi kecil untuk callback dan penjadwalan pada event loop Tk.
`* pada parameter`	Argumen setelah * harus diberikan dengan nama, misalnya context=....

Contoh runnable mencantumkan import yang diperlukan. Contoh lain menyebut konteks db, root, frame, atau objek service; variabel itu disediakan aplikasi atau fixture tes. Gunakan database sementara untuk eksperimen. Aplikasi tidak perlu diubah untuk membaca dokumen ini.

Daftar isi

Bagian A - Fondasi, tanggal, lokasi, dan database **4**
01. Menghitung umur dan status barang . . . 4
02. Membentuk alamat dan 27 lokasi per rak . . . 6
03. Memilih jalur data source dan executable . . . 8
04. Membuka SQLite dan mengendalikan migrasi . . . 10
05. Backup migrasi dan seed satu kali . . . 12

Bagian B - Barang, QR, impor, sesi, dan stok **14**
06. Memvalidasi formulir dan menambahkan batch . . . 14
07. Mengedit stok dengan pemeriksaan nilai lama dan menyaring inventory . . . 16
08. Mengubah ID batch menjadi QR dan membaca data terkini . . . 18
09. Membaca Excel, meninjau kesalahan, lalu menyimpan impor atomik . . . 20
10. Memulai sesi dan menjaga snapshot saat QR dipindai ulang . . . 22
11. Menyimpan catatan dan menyelesaikan sesi yang terkunci . . . 24
12. Mengeluarkan stok sekali saja dalam transaksi yang utuh . . . 26

Bagian C - Rekomendasi dan keputusan petugas **28**
13. Membaca kebutuhan material dengan JOIN . . . 28
14. Menyaring site dan menghitung jumlah usulan . . . 30
15. Customer dan urutan cabang rekomendasi . . . 32
16. Menyimpan keputusan secara atomik . . . 34
17. Penyelesaian terpisah dan proyeksi histori . . . 36

Bagian D - UI, kamera, laporan, dan executable **38**
18. Navigasi Tkinter dan pemeriksaan pergantian hari . . . 38
19. Agregasi section dan pembukaan tampak samping rak . . . 40
20. Decode QR, antrean frame, dan pelepasan perangkat kamera . . . 42
21. LiveMonitor: konfirmasi dua poll dan masa tahan QR . . . 44
22. Membaca laporan dari snapshot sesi . . . 46
23. Ekspor PDF: file sementara, font, dan teks aman . . . 48
24. Entry point, build Windows, dan path aplikasi portable . . . 50

Bagian A / Fondasi, tanggal, lokasi, dan database

01. Menghitung umur dan status barang

Fungsi aging() menjadi aturan bersama untuk inventory, sesi, dashboard, dan rekomendasi. Rumus ditempatkan di satu modul agar batas umur tidak dihitung berbeda pada setiap halaman.

Fungsi aging() lengkap

Sumber: warehouse/domain.py : 26-33 | aging

```
26  def aging(received, today=None, normal_max=NORMAL_MAX, age_limit=AGE_LIMIT):
27      if not 0 <= normal_max < age_limit:
28          raise ValueError("Batas aging tidak valid.")
29      entered = date.fromisoformat(received)
30      days = ((today or date.today()) - entered).days
31      if days < 0:
32          raise ValueError("Tanggal masuk berada di masa depan.")
33      return days, "Normal" if days <= normal_max else "Warning" if days <= age_limit else
33  > "Melewati batas"
```

`def aging(...)`	def mendefinisikan fungsi. received wajib diisi; today=None berarti tanggal acuan boleh tidak diberikan. normal_max dan age_limit mengambil konstanta default 30 dan 90.
`0 <= normal_max < age_limit`	Perbandingan berantai memastikan batas Normal lebih kecil dari batas akhir. not membalik hasil; raise menghentikan fungsi dengan ValueError jika aturan tidak valid.
`date.fromisoformat(received)`	Mengubah teks tanggal YYYY-MM-DD menjadi objek date. Tanggal kalender atau format yang salah menyebabkan ValueError.
`today or date.today()`	or memilih today bila bernilai benar; jika None, memakai tanggal komputer. Parameter today memungkinkan tes dengan tanggal tetap.
`(... - entered).days`	Pengurangan dua tanggal menghasilkan timedelta. Properti .days mengambil jumlah hari sebagai integer.
`if days < 0`	Tanggal penerimaan masa depan ditolak, sehingga tidak menjadi umur negatif yang dianggap aman.
`return days, ...`	Koma membentuk tuple dua nilai: umur dan status. Ekspresi if/else satu baris mengecek Normal dahulu, kemudian Warning, lalu Melewati batas.

Input dan hasil

`Input`	received berupa teks ISO; today berupa date opsional; batas default 30/90.
`Output`	Tuple (jumlah_hari, status), misalnya (31, "Warning").

Contoh penggunaan

Contoh dengan import lengkap.

```
from datetime import date
from warehouse.domain import aging

print(aging("2026-09-04", today=date(2026, 10, 5)))
```

Hasil yang diharapkan: (31, "Warning")

Hubungan dengan fitur lain

Inventory.list_batches() memakai batas default. Sessions.add_code() memakai batas dari sesi dan menyimpan hasilnya sebagai snapshot.

Kondisi error

- ValueError untuk batas tidak valid, tanggal tak valid, atau penerimaan masa depan.

Catatan implementasi

- Hari ke-30 masih Normal, hari ke-90 masih Warning; Melewati batas mulai hari ke-91. Status umur tidak menilai kelayakan fisik.

Referensi tes: test_foundation.py, test_sessions.py

02. Membentuk alamat dan 27 lokasi per rak

location_code() menyusun identitas fisik dan memeriksa batas sumbu. Locations.create_rack() memakai tiga loop untuk mendaftarkan semua kombinasi K/R/T.

Format dan validasi alamat

Sumber: warehouse/domain.py : 9-14 | location_code

```
9  def location_code(area, rack, column, row, level):
10     if not re.fullmatch(r"[A-Z]", area) or not re.fullmatch(r"[A-Z][0-9]{2}", rack):
11         raise ValueError("Area harus A–Z dan rak seperti B03.")
12     if any(type(value) is not int or not 1 <= value <= 3 for value in (column, row, level)):
13         raise ValueError("Kolom, baris, dan tingkat harus 01–03.")
14     return f"{area}-{rack}-K{column:02d}-R{row:02d}-T{level:02d}"
```

`re.fullmatch(...)`	Regex harus cocok dengan seluruh teks, bukan hanya sebagian. [A-Z] menerima satu huruf area; [A-Z][0-9]{2} menerima rak seperti B03.
`any(...)`	any bernilai True jika ada satu koordinat yang bukan integer tepat atau di luar 1-3. Generator mengecek kolom, baris, dan tingkat tanpa membuat list tambahan.
`type(value) is not int`	Pemeriksaan tipe tepat menolak bool, walaupun bool merupakan turunan int dalam Python.
`f"...{column:02d}..."`	f-string memasukkan nilai ke string. :02d menampilkan integer sebagai dua digit, misalnya 3 menjadi 03.

Tiga loop dan INSERT OR IGNORE

Sumber: warehouse/locations.py : 42-51 | Locations.create_rack

```
42 codes = []
43 with self.db:
44     for column in range(1, 4):
45         for row in range(1, 4):
46             for level in range(1, 4):
47                 code = location_code(area, rack, column, row, level)
48                 self.db.execute('''INSERT OR IGNORE INTO
48 > locations(code,area,rack,column,row,level)
49                     VALUES(?,?,?,?,?,?)''', (code, area, rack, column, row, level))
50                 codes.append(code)
51 return codes
```

`with self.db`	Context manager koneksi mengelola commit bila berhasil atau rollback bila exception keluar dari blok. Ia tidak menutup koneksi.
`range(1, 4)`	Batas akhir tidak ikut: menghasilkan 1,2,3. Tiga loop memberi 3 x 3 x 3 = 27 kombinasi.
`INSERT OR IGNORE`	Alamat yang sudah ada tidak ditambahkan ulang; ini memungkinkan melengkapi rak parsial. Constraint tetap berada di database.
`VALUES(?,?,?,?,?,?)`	Enam placeholder diisi tuple parameter. Nilai data tidak digabungkan langsung ke teks SQL.

Input dan hasil

`Input`	Area dan rak valid; untuk satu alamat, column,row,level berupa integer 1-3.
`Output`	location_code() menghasilkan string; create_rack() mengembalikan list 27 kode.

Contoh penggunaan

Contoh dengan import lengkap.

```
from warehouse.domain import location_code
print(location_code("A", "B03", 2, 3, 1))
```

Hasil yang diharapkan: A-B03-K02-R03-T01

Hubungan dengan fitur lain

Form lokasi memanggil Locations; form barang hanya menerima alamat yang terdaftar. Denah mengagregasi K/R dan menampilkan tingkat pada detail kolom.

Kondisi error

- ValueError untuk area/rak/sumbu yang tidak valid. location_code() sendiri tidak menormalisasi huruf kecil; layanan parse_location() melakukan normalisasi.

Catatan implementasi

- Area, rak, kolom, baris, tingkat adalah koordinat terpisah. Tidak ada pemeriksaan kapasitas atau posisi fisik otomatis dari kamera.

Referensi tes: test_locations.py, test_locations_ui.py

03. Memilih jalur data source dan executable

runtime_paths.py memisahkan aset bundled dari data yang dapat diubah. Database mengikuti executable saat aplikasi dikemas, sehingga tidak bergantung pada folder tempat pengguna menjalankan perintah.

Akar aset modul

Sumber: warehouse/runtime_paths.py : 6-8 | resource_root

```
6  def resource_root():
7      """PyInstaller keeps package files and assets together in _internal."""
8      return Path(__file__).resolve().parent.parent
```

`__file__`	Variabel khusus Python berisi jalur file modul yang sedang dijalankan. Pada paket, modul berada dalam _internal.
`Path(...).resolve()`	Path mengubah teks menjadi objek jalur. resolve() menghasilkan jalur absolut dengan tautan yang diselesaikan.
`.parent.parent`	Naik dua direktori dari file runtime_paths.py menuju akar resource.

Percabangan source dan frozen

Sumber: warehouse/runtime_paths.py : 11-15 | application_root

```
11 def application_root():
12     """Data follows the executable, never the launcher's working directory."""
13     if getattr(sys, 'frozen', False):
14         return Path(sys.executable).resolve().parent
15     return resource_root()
```

`getattr(sys, "frozen", False)`	getattr membaca atribut; jika tidak ada, hasil default False. PyInstaller menandai aplikasi bundled sebagai frozen.
`sys.executable`	Pada source ini interpreter Python; dalam cabang frozen menunjuk WarehouseCheck.exe. .parent mengambil folder executable.
`return resource_root()`	Dalam mode source, akar data memakai akar proyek yang sama dengan resource.

Operator jalur Path

Sumber: warehouse/runtime_paths.py : 18-19 | database_path

```
18 def database_path():
19     return application_root() / 'data' / 'warehouse_demo.db'
```

`application_root() / "data" / ...`	Operator / pada Path menyambung komponen direktori; bukan pembagian angka.

Input dan hasil

`Input`	Tidak ada argumen; membaca lingkungan proses.
`Output`	Objek pathlib.Path, bukan string filename relatif working directory.

Contoh penggunaan

Contoh dengan import lengkap.

```
from warehouse.runtime_paths import database_path
print(database_path().name)
```

Hasil yang diharapkan: warehouse_demo.db

Hubungan dengan fitur lain

database.DEMO_PATH memakai database_path(). UI memakai path itu secara default; tes dapat memasok path sementara.

Kondisi error

- PermissionError/OSError muncul saat pemakai path mencoba menulis pada direktori yang tidak dapat ditulis, bukan saat sekadar membuat objek Path.

Catatan implementasi

- Database di data/ dan di folder paket adalah dua file terpisah. Mengedit source tidak memperbarui executable atau data paket secara otomatis.

Referensi tes: tests/test_runtime_paths.py

04. Membuka SQLite dan mengendalikan migrasi

connect() membuat koneksi sekaligus memastikan database milik aplikasi dan schema sesuai. Setiap pemanggil mendapatkan sqlite3.Row sehingga hasil query bisa dibaca memakai nama kolom.

Awal connect(): koneksi dan pemeriksaan versi

Sumber: warehouse/database.py : 168-178 | connect

```
168 def connect(path=DEMO_PATH):
169     path = Path(path)
170     path.parent.mkdir(parents=True, exist_ok=True)
171     db = sqlite3.connect(path)
172     db.row_factory = sqlite3.Row
173     db.execute("PRAGMA foreign_keys=ON")
174     db.execute("PRAGMA busy_timeout=5000")
175     try:
176         version = db.execute("PRAGMA user_version").fetchone()[0]
177         if version > SCHEMA_VERSION:
178             raise ValueError("Database berasal dari aplikasi yang lebih baru.")
```

`path.parent.mkdir(parents=True, exist_ok=True)`	Membuat folder induk beserta induknya; tidak error bila folder sudah ada. Parameter ini tidak menghapus isi lama.
`sqlite3.connect(path)`	Membuka atau membuat file SQLite. Koneksi dikembalikan oleh fungsi setelah migrasi berhasil.
`db.row_factory = sqlite3.Row`	Hasil SELECT dapat diakses row["nama_kolom"] dan diubah dengan dict(row).
`PRAGMA foreign_keys=ON`	Mengaktifkan pemeriksaan FK pada koneksi ini agar relasi tidak merujuk identitas yang hilang.
`PRAGMA busy_timeout=5000`	Menunggu lock sampai sekitar 5000 milidetik sebelum kegagalan; bukan dukungan multiuser penuh.
`fetchone()[0]`	execute menjalankan query; fetchone mengambil satu row; [0] mengambil nilai versi pertama.
`if version > SCHEMA_VERSION`	Menolak database dari program lebih baru; kode lama tidak boleh menebak schema itu.

Penanda format dan backup sebelum migrasi

Sumber: warehouse/database.py : 201-205 | connect

```
201 mode = db.execute("SELECT value FROM metadata WHERE key='mode'").fetchone()
202 if not mode or mode[0] != "demo":
203     raise ValueError("File ini bukan database demo.")
204 if 0 < version < SCHEMA_VERSION:
205     _backup_before_migration(db, path, version)
```

`WHERE key='mode'`	Penanda mode=demo memeriksa format database aplikasi. Ini bukan autentikasi pengguna.
`0 < version < SCHEMA_VERSION`	Backup hanya untuk database lama yang sudah berversi. Database baru versi 0 tidak memiliki riwayat yang perlu dicadangkan.

`_backup_before_migration(...)`	Memanggil helper sebelum rangkaian migrasi; tiap tahap setelahnya memiliki transaksi sendiri.

Input dan hasil

`Input`	path berupa Path atau string; default DEMO_PATH.
`Output`	Koneksi SQLite yang sudah siap, dengan schema 7 pada kode ini. Pemanggil tetap harus menutupnya.

Contoh penggunaan

Contoh dengan import lengkap.

```
from pathlib import Path
from tempfile import TemporaryDirectory
from warehouse.database import connect

with TemporaryDirectory() as folder:
    db = connect(Path(folder) / "uji.db")
    try:
        print(db.execute("PRAGMA user_version").fetchone()[0])
    finally:
        db.close()
```

Hasil yang diharapkan: 7; database percobaan dibersihkan setelah blok selesai.

Hubungan dengan fitur lain

Application membuka koneksi sekali. Service inventory/sesi/rekomendasi berbagi koneksi tersebut pada thread UI.

Kondisi error

- ValueError untuk schema lebih baru, database versi0 yang sudah punya tabel asing, atau metadata format tidak cocok. sqlite3.Error untuk kegagalan file/SQL.

Catatan implementasi

- Migrasi v4 merombak alamat; v5 keputusan, v6 konteks kebutuhan, v7 tujuan Site/Customer. Detail SQL lengkap ada pada file sumber dan panduan data.

Referensi tes: test_foundation.py, tests/test_recommendation_data.py

05. Backup migrasi dan seed satu kali

Backup memakai API SQLite agar isi yang sudah committed ikut tercakup. Seed minimal tidak dijalankan berulang, sehingga penerimaan dummy tidak bergeser setiap startup.

Backup database lengkap

Sumber: warehouse/database.py : 12-21 | _backup_before_migration

```
12  def _backup_before_migration(db, path, version):
13      """Keep a complete SQLite backup, including committed WAL content."""
14      stamp = datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%S%fZ')
15      backup_path = path.with_name(f'{path.name}.pre-v{version}-{stamp}.bak')
16      backup = sqlite3.connect(backup_path)
17      try:
18          db.backup(backup)
19      finally:
20          backup.close()
21      return backup_path
```

`datetime.now(timezone.utc).strftime(...)`	Membuat timestamp UTC untuk nama backup yang unik hingga mikrodetik; ini penamaan file, bukan waktu aging.
`path.with_name(...)`	Mengganti nama terakhir sambil mempertahankan direktori database. Backup diletakkan di sebelah file asal.
`db.backup(backup)`	Menyalin database melalui SQLite, termasuk transaksi committed yang berada pada WAL bila ada.
`try ... finally`	finally dijalankan meskipun penyalinan gagal, sehingga koneksi backup selalu ditutup.

Penanda agar seed tidak diulang

Sumber: warehouse/database.py : 356-358 | seed_demo

```
356  with db:
357      if db.execute("SELECT value FROM metadata WHERE key='seed_date'").fetchone():
358          return
```

`with db`	Pengisian seed berlangsung dalam satu transaksi koneksi.
`fetchone() lalu return`	Jika seed_date sudah ada, fungsi keluar sebelum menambah lokasi atau batch lagi.

Input dan hasil

`Input`	Helper backup: koneksi sumber, path, versi asal. Seed: koneksi schema aplikasi dan tanggal opsional.
`Output`	Backup mengembalikan Path. seed_demo() tidak memberi nilai hasil; efeknya menambah data hanya sekali.

Contoh penggunaan

Contoh dengan import lengkap.

```
from pathlib import Path
from tempfile import TemporaryDirectory
from datetime import date
from warehouse.database import connect, seed_demo

with TemporaryDirectory() as folder:
    db = connect(Path(folder) / "uji.db")
    try:
        seed_demo(db, today=date(2026, 10, 5))
        seed_demo(db, today=date(2026, 10, 5))
        print(db.execute("SELECT COUNT(*) FROM batches").fetchone()[0])
    finally:
        db.close()
```

Hasil yang diharapkan: 50, bukan 100: pemanggilan kedua tidak menggandakan seed.

Hubungan dengan fitur lain

connect() memanggil backup sebelum migrasi. Application memanggil seed_demo() setelah connect(). Builder rilis membuat data awal tersendiri dari seed200.

Kondisi error

- OSError/sqlite3.Error bila file backup tidak dapat dibuat. Seed bukan operasi untuk mereset database atau menghapus riwayat.

Catatan implementasi

- Seed minimal: 50 batch/162 alamat. Paket rilis: 200 batch/324 alamat dengan skenario Site/Customer. Tanggal seed tetap; aging bertambah mengikuti tanggal komputer.

Referensi tes: test_foundation.py, test_package_build.py

Bagian B / Barang, QR, impor, sesi, dan stok

06. Memvalidasi formulir dan menambahkan batch

Inventory.validate membersihkan masukan sebelum penulisan. Inventory.add membuat identitas penerimaan baru; dua penerimaan boleh memiliki material dan tanggal sama. Kesalahan membawa field agar formulir menunjukkan masukan bermasalah.

Membersihkan teks dan memastikan lokasi terdaftar

Sumber: warehouse/inventory.py : 18-31 | Inventory.validate

```
18  def validate(self, values):
19      data = {}
20      for field, label in (("material", "Material"), ("description", "Deskripsi"),
21                           ("plant", "Plant"), ("storage_location", "Storage Location")):
22          value = str(values.get(field, "")).strip()
23          if not value or len(value) > 200:
24              raise BatchValidationError(field, f"{label} wajib diisi, maksimal 200 karakter.")
25          data[field] = value
26      code = str(values.get("location_code", "")).strip().upper()
27      try:
28          Locations(self.db).require(code)
29      except ValueError as exc:
30          raise BatchValidationError("location_code", str(exc)) from exc
31      data["location_code"] = code
```

`def validate(self, values):`	self menunjuk objek Inventory; values adalah mapping masukan formulir.
`for field, label in (...):`	Setiap pasangan tuple dibongkar menjadi nama field dan label pesan.
`str(values.get(field, "")).strip()`	get memberi default kosong; str mengubah tipe; strip membuang spasi tepi.
`if not value or len(value) > 200:`	or menolak teks kosong maupun teks melebihi 200 karakter.
`Locations(self.db).require(code)`	Objek layanan memakai koneksi sama dan memeriksa lokasi yang tersedia.
`raise BatchValidationError(...) from exc`	Exception baru membawa field terkait sekaligus menyimpan penyebab aslinya.

Menulis penerimaan baru dalam transaksi

Sumber: warehouse/inventory.py : 46-54 | Inventory.add

```
46  def add(self, values):
47      data = self.validate(values)
48      batch_id = "BAT-" + uuid4().hex.upper()
49      with self.db:
50          self.db.execute("""INSERT INTO batches
51              (id,material,description,plant,storage_location,location_code,received_date,stock)
52              VALUES (:id,:material,:description,:plant,:storage_location,:location_code,:receiv
52  > ed_date,:stock)""",
53              dict(data, id=batch_id))
```

```
54        return batch_id
```

`data = self.validate(values)`	Pemanggilan method harus berhasil sebelum ada penulisan database.
`uuid4().hex.upper()`	UUID acak menjadi 32 digit heksadesimal kapital sesudah awalan BAT-.
`with self.db:`	Context manager SQLite melakukan commit saat sukses, rollback saat gagal.
`VALUES (:id,:material,...)`	Placeholder bernama mengikat nilai dictionary tanpa merangkai teks masukan sebagai SQL.
`dict(data, id=batch_id)`	Menyalin data bersih sambil menambahkan kunci id untuk INSERT.
`return batch_id`	Mengembalikan string ID yang dapat dipakai label dan pencarian berikutnya.

Input dan hasil

`Input`	values: material, description, plant, storage_location, location_code, received_date, stock.
`Output`	validate: dict bersih dengan stock bertipe int; add: string BAT- diikuti 32 digit heksadesimal.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
1  from datetime import date
2  from warehouse.inventory import Inventory
3  values = dict(material='00072', description='Test part', plant='BHT',
4      storage_location='PBHT', location_code='A-B01-K01-R01-T01',
5      received_date=date.today().isoformat(), stock='12')
6  key = Inventory(db).add(values)
7  print(Inventory(db).list_batches(query=key)[0]['stock'])
```

Hasil yang diharapkan: Mencetak 12. Material tetap '00072'; key berupa ID acak BAT-..., dan satu batch baru tersimpan.

Hubungan dengan fitur lain

batch_ui mengambil snapshot masukan, memanggil add, lalu memakai ID hasil untuk memperbarui tampilan.

Kondisi error

- Tanggal harus valid YYYY-MM-DD dan tidak di masa depan; stok harus digit 0–999999999; lokasi wajib terdaftar.

Catatan implementasi

- Contoh memerlukan db berskema aplikasi dan lokasi tersebut. Lanjutan validate memeriksa tanggal serta mengubah stok menjadi int.

Referensi tes: test_inventory.py

07. Mengedit stok dengan pemeriksaan nilai lama dan menyaring inventory

Inventory.update menjaga ID sambil memvalidasi data pengganti. expected_stock mencegah formulir lama menimpa stok setelah transaksi barang keluar. list_batches menambahkan umur/status, lalu menerapkan beberapa filter bersama.

UPDATE bersyarat dan penolakan formulir lama

Sumber: warehouse/inventory.py : 56-71 | Inventory.update

```
56  def update(self, batch_id, values, expected_stock=None):
57      data = self.validate(values)
58      with self.db:
59          condition='id=:id'
60          if expected_stock is not None:
61              condition+=' AND stock=:expected_stock'
62              data['expected_stock']=expected_stock
63          result = self.db.execute("""UPDATE batches SET
63  > material=:material,description=:description,
64              plant=:plant,storage_location=:storage_location,location_code=:location_code,
65              received_date=:received_date,stock=:stock WHERE """+condition, dict(data,
65  > id=batch_id))
66          if result.rowcount != 1:
67              current=self.db.execute('SELECT stock FROM batches WHERE
67  > id=?',(batch_id,)).fetchone()
68              if current is not None:
69                  raise BatchValidationError('stock',f'Stok sudah berubah menjadi {current[0]}
69  > unit. Tutup formulir dan buka ulang Edit barang.')
70              raise ValueError("Batch tidak ditemukan. Muat ulang Data Barang.")
71      return batch_id
```

`expected_stock=None`	Parameter opsional: None berarti pemanggil tidak meminta pemeriksaan stok sebelumnya.
`condition='id=:id'`	Kondisi dasar memilih batch lewat ID, bukan material atau lokasi.
`condition+=' AND stock=:expected_stock'`	Operator += menambahkan syarat stok pada perintah UPDATE yang sama.
`data['expected_stock']=expected_stock`	Nilai pembanding ikut dikirim melalui placeholder bernama, bukan interpolasi.
`if result.rowcount != 1:`	Jumlah baris berbeda dari satu menandakan stok berubah atau batch hilang.
`current[0]`	Indeks nol mengambil stok saat ini untuk pesan kesalahan yang konkret.

Menghitung status dan menggabungkan filter

Sumber: warehouse/inventory.py : 78-88 | Inventory.list_batches

```
78  try:
79      item["age"], item["status"] = aging(item["received_date"], today=today)
80  except ValueError:
81      item["age"], item["status"] = "—", "Data perlu diperbaiki"
82  haystack = " ".join(str(item[k]) for k in ("id", "material", "description", "location_code",
82  > "plant", "storage_location"))
```

```
if (query.strip().casefold() in haystack.casefold()
        and (not status or item["status"] == status)
        and (not location or item["location_code"] == location)
        and (availability!='active' or item['stock']>0)
        and (availability!='empty' or item['stock']==0)):
    result.append(item)
```

`item["age"], item["status"] = aging(...)`	Tuple hasil aging dibongkar menjadi dua field tambahan pada dictionary.
`except ValueError:`	Tanggal rusak ditampilkan sebagai data perlu diperbaiki; daftar tetap dibaca.
`" ".join(str(item[k]) for k in (...))`	Generator mengubah beberapa field menjadi satu teks pencarian gabungan.
`query.strip().casefold() in haystack.casefold()`	Pencarian substring mengabaikan spasi tepi dan perbedaan huruf besar kecil.
`and (not status or ...)`	Semua filter aktif harus cocok; string kosong menonaktifkan filter tersebut.
`availability!='empty' or item['stock']==0`	Filter empty hanya menerima stok nol; active hanya stok positif.

Input dan hasil

`Input`	update: batch_id, values lengkap, expected_stock opsional. list_batches: today, query, status, location, availability.
`Output`	update mengembalikan ID sama; list_batches mengembalikan list dict batch beserta area/rak/kolom/baris/tingkat, age, status.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
from warehouse.inventory import Inventory
service = Inventory(db)
row = service.list_batches(query='DEMO-0002')[0]
service.update(row['id'], dict(row, stock='0'), expected_stock=row['stock'])
print(service.list_batches(query='DEMO-0002', availability='empty')[0]['stock'])
```

Hasil yang diharapkan: Pada database demo baru, mencetak 0. ID tetap DEMO-0002; filter active kemudian menghasilkan list kosong untuk ID itu.

Hubungan dengan fitur lain

batch_ui memasok stok awal sebagai expected_stock; layar inventory membaca list_batches kembali sesudah penyimpanan.

Kondisi error

- Batch hilang memunculkan ValueError; stok yang telah berubah memunculkan BatchValidationError pada field stock.

Catatan implementasi

- Contoh memerlukan db demo baru. today menentukan tanggal acuan status tanpa menyimpan ulang batch.

Referensi tes: test_inventory.py

08. Mengubah ID batch menjadi QR dan membaca data terkini

Label menyimpan identitas stabil ID:<batch_id>. resolve mencari ID secara tepat saat pemindaian; perubahan deskripsi, lokasi, atau stok tidak menuntut penggantian QR. qr_image menggambar payload yang sama.

Memvalidasi awalan dan mencari ID secara tepat

Sumber: warehouse/labels.py : 14-24 | resolve

```
14  def resolve(db, value):
15      """Exact ID lookup; never infer batch from material or location."""
16      if not isinstance(value, str) or not value.startswith('ID:'):
17          raise ValueError('QR bukan identitas batch Warehouse Check.')
18      key = value[3:]
19      if payload(key) != value:
20          raise ValueError('Isi QR tidak valid.')
21      row = db.execute('SELECT * FROM batches WHERE id=?', (key,)).fetchone()
22      if row is None:
23          raise ValueError('Batch QR tidak ditemukan di database ini.')
24      return dict(row)
```

`isinstance(value, str)`	Memeriksa tipe sebelum operasi string agar masukan bukan teks ditolak.
`value.startswith('ID:')`	Awalan wajib membedakan identitas batch dari material atau kode lokasi.
`key = value[3:]`	Slice mengambil seluruh karakter mulai indeks tiga, setelah awalan ID:.
`if payload(key) != value:`	Pembentukan ulang payload memeriksa konsistensi format dan validitas ID.
`(key,)`	Koma membentuk tuple satu elemen sebagai parameter placeholder SQL ?.
`fetchone(); return dict(row)`	Satu record dicari; hasil SQLite diubah menjadi dictionary field batch.

Menghasilkan gambar QR dari payload batch

Sumber: warehouse/labels.py : 27-31 | qr_image

```
27  def qr_image(batch_id):
28      qr = qrcode.QRCode(error_correction=qrcode.constants.ERROR_CORRECT_M, box_size=10,
28  > border=4)
29      qr.add_data(payload(batch_id))
30      qr.make(fit=True)
31      return qr.make_image(fill_color='black', back_color='white').convert('RGB')
```

`def qr_image(batch_id):`	Fungsi menerima satu ID dan mengembalikan objek gambar Pillow.
`qrcode.QRCode(error_correction=..., box_size=10, border=4)`	Argumen bernama memilih koreksi M, modul 10 piksel, dan tepi empat modul.
`qr.add_data(payload(batch_id))`	Pemanggilan bersarang membentuk teks identitas sebelum memasukkannya ke QR.
`qr.make(fit=True)`	Library memilih ukuran QR yang dapat menampung teks tersebut.

`make_image(...).convert('RGB')`	Method berantai menghasilkan gambar hitam putih dalam mode warna RGB.

Input dan hasil

`Input`	payload dan qr_image: batch_id string; resolve: koneksi db dan teks berawalan ID:.
`Output`	payload: string identitas; resolve: dict batch terkini; qr_image: PIL.Image RGB, tanpa menyimpan file otomatis.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
from warehouse.labels import payload, resolve, qr_image
code = payload('DEMO-0001')
item = resolve(db, code)
image = qr_image(item['id'])
print(code, item['id'], image.mode)
```

Hasil yang diharapkan: Mencetak ID:DEMO-0001 DEMO-0001 RGB. Gambar dapat dipindai kembali menjadi ID:DEMO-0001.

Hubungan dengan fitur lain

label_ui memanggil payload/resolve; scanner dan Sessions.add_code memakai resolve untuk identitas sama.

Kondisi error

- ID kosong, memiliki spasi tepi, bukan string, atau melebihi 100 karakter ditolak oleh payload; batch tidak ditemukan juga ditolak.

Catatan implementasi

- Contoh membutuhkan db demo. QR berisi ID, sehingga label lama tetap menemukan data master terbaru sesudah edit dan aplikasi dibuka ulang.

Referensi tes: test_labels.py

09. Membaca Excel, meninjau kesalahan, lalu menyimpan impor atomik

read_inventory membaca sheet Inventory menjadi record masukan. preview mengumpulkan errors/notices tanpa penulisan. commit_import mengunci database, mengulang preview, kemudian menyimpan seluruh baris dalam satu transaksi.

Validasi baris dan deteksi data identik

Sumber: warehouse/inventory_excel.py : 97-108 | preview

```
97      try:
98          clean = inventory.validate(values)
99          signature = fingerprint(dict(clean,id=key))
100         if not key and (signature in existing_data or signature in seen_data):
101             errors.append('Data identik terdeteksi. Jika penerimaan berbeda, isi Batch ID unik
101 > secara eksplisit.')
102         seen_data.add(signature)
103     except ValueError as exc:
104         errors.append(str(exc))
105         clean = values
106     if key:seen_ids.add(key)
107     result.append(dict(row=record['row'], values=dict(clean,id=key), errors=errors,
107 > notices=notices))
108  return result
```

`clean = inventory.validate(values)`	Aturan formulir yang sama dipakai kembali untuk tiap baris Excel.
`fingerprint(dict(clean,id=key))`	Tuple field selain ID, setelah strip/casefold, menjadi penanda data identik.
`not key and (signature in existing_data or signature in seen_data)`	Jika Batch ID kosong, fingerprint yang sudah ada di database atau file ditolak. ID eksplisit yang unik memungkinkan penerimaan berbeda dengan data identik.
`except ValueError as exc:`	Kesalahan masuk list errors sehingga peninjauan dapat menampilkan semua baris.
`result.append(dict(row=..., values=..., errors=..., notices=...))`	Setiap hasil mempertahankan nomor baris Excel dan rincian pemeriksaannya.

Mengunci, memeriksa ulang, dan menulis semua batch

Sumber: warehouse/inventory_excel.py : 115-129 | commit_import

```
115  db.execute('BEGIN IMMEDIATE')
116  try:
117      checked = preview(db, source)
118      if any(r['errors'] for r in checked):
119          raise ValueError('Impor dibatalkan: ada kesalahan/duplikasi. Muat ulang preview untuk
119 > melihat detail.')
120      keys = []
121      for row in checked:
122          values = dict(row['values'])
123          values['id'] = values['id'] or 'BAT-' + uuid4().hex.upper()
124          db.execute('''INSERT INTO batches
```

```
            (id,material,description,plant,storage_location,location_code,received_date,stock)
            VALUES (:id,:material,:description,:plant,:storage_location,:location_code,:receiv
> ed_date,:stock)''', values)
        keys.append(values['id'])
    db.commit()
    return keys
```

`db.execute('BEGIN IMMEDIATE')`	Meminta kunci penulisan sebelum membaca ulang keadaan database.
`checked = preview(db, source)`	Preview di layar bukan izin tetap; validasi diulang saat penyimpanan.
`any(r['errors'] for r in checked)`	Generator dan any mendeteksi sedikitnya satu baris yang memiliki kesalahan.
`values['id'] = values['id'] or 'BAT-' + ...`	Operator or mempertahankan ID eksplisit atau membuat ID untuk kolom kosong.
`db.commit(); return keys`	Seluruh INSERT disahkan bersama; hasil adalah list ID batch yang disimpan.

Input dan hasil

`Input`	read_inventory: path .xlsx. preview/commit_import: db dan list record {row, values, issues} hasil pembacaan.
`Output`	Pembacaan: list record; preview: list {row, values, errors, notices}; commit_import: list ID tersimpan.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
from warehouse.inventory_excel import read_inventory, preview, commit_import
source = read_inventory('penerimaan.xlsx')
checked = preview(db, source)
if not any(row['errors'] for row in checked):
    print(commit_import(db, source))
```

Hasil yang diharapkan: Untuk satu baris valid ber-ID EXT-002 dan stok 3, mencetak ['EXT-002']. Impor file sama berikutnya ditolak sebagai duplikat.

Hubungan dengan fitur lain

excel_ui membaca file di worker, menampilkan preview, lalu meneruskan source asli ke commit_import.

Kondisi error

- Header hilang/duplikat, sheet salah, lebih dari 5000 baris, rumus/error Excel, kode numerik, atau validasi baris gagal menghalangi impor. Urutan header boleh berbeda; heuristik nilai bergeser menandai lokasi mirip tanggal dengan stok kosong.

Catatan implementasi

- Contoh memerlukan file template berisi baris valid dan db. read_inventory menutup workbook melalui finally; commit_import melakukan rollback lalu raise bila gagal. Lokasi lama diterjemahkan saat preview.

Referensi tes: test_inventory_excel.py

10. Memulai sesi dan menjaga snapshot saat QR dipindai ulang

Sessions.start menyimpan petugas, area, waktu, dan batas umur sesi Draft. add_code merekam satu hasil per sesi/batch. Pemindaian ulang mempertahankan snapshot pertama beserta waktu, lokasi, dan rekomendasi pengecekan.

Validasi petugas, area, dan batas umur sesi

Sumber: warehouse/sessions.py : 21-27 | Sessions.start

```
21  def start(self,officer,area,normal_max=NORMAL_MAX,age_limit=AGE_LIMIT):
22      officer,area=required(officer,'Nama / NIK petugas'),required(area,'Area pengecekan')
23      try:
24          normal_max,age_limit=int(str(normal_max)),int(str(age_limit))
25          if not 0<=normal_max<age_limit<=365000:raise ValueError()
26      except ValueError as exc:
27          raise ValueError('Batas umur harus bilangan bulat: 0 ≤ normal < batas umur ≤ 365000.')
27  > from exc
```

`normal_max=NORMAL_MAX,age_limit=AGE_LIMIT`	Parameter default mengambil batas aplikasi; pemanggil dapat menentukan batas sesi.
`officer,area=required(...),required(...)`	Dua hasil fungsi dibongkar sekaligus setelah pembersihan teks wajib.
`int(str(normal_max))`	Konversi mengizinkan string bilangan bulat; teks desimal seperti 1.5 gagal.
`0<=normal_max<age_limit<=365000`	Perbandingan berantai memastikan batas normal lebih kecil dari batas umur.
`raise ValueError(...) from exc`	Pesan validasi yang mudah dibaca diterbitkan bersama penyebab konversinya.

Mendeteksi scan ulang lalu menyiapkan snapshot pemeriksaan

Sumber: warehouse/sessions.py : 64-78 | Sessions.add_code

```
64  batch=code[3:]
65  exists=self.db.execute('SELECT 1 FROM check_results WHERE session_id=? AND
65  > batch_id=?',(key,batch)).fetchone()
66  if exists:
67      snapshot=self.db.execute('SELECT snapshot FROM check_results WHERE session_id=? AND
67  > batch_id=?',(key,batch)).fetchone()[0]
68      self.mark_check(key,batch,json.loads(snapshot)['location_code'])
69      return batch,False
70  item=resolve(self.db,code)
71  moment=datetime.now().astimezone()
72  age,status=aging(item['received_date'],today=moment.date(),normal_max=session['normal_max'],ag
72  > e_limit=session['age_limit'])
73  context=recommendation_context(self.db,item['material'])
74  if not context['sites'] and not context['customers']:
75      context=None
76  advice=recommend_for(item,normal_max=session['normal_max'],
77                       age_limit=session['age_limit'],today=moment.date(),context=context)
78  snapshot=dict(item,recommendation_at_check=advice)
```

`batch=code[3:]`	Slice menghapus awalan ID: setelah kode dinormalisasi pada awal method.
`WHERE session_id=? AND batch_id=?`	Dua parameter mencari duplikasi hanya dalam sesi yang sedang dikerjakan.
`json.loads(snapshot)['location_code']`	JSON historis dibaca kembali untuk memperbarui penanda aktivitas dari lokasi lama.
`return batch,False`	Tuple False menyatakan pemindaian ulang tanpa menambahkan hasil baru.
`aging(...normal_max=session['normal_max'],...)`	Umur/status dihitung pada hari scan memakai batas yang tersimpan di sesi.
`snapshot=dict(item,recommendation_at_check=advice)`	Salinan data batch ditambah rekomendasi sebelum diserialisasi sebagai rekaman historis.

Input dan hasil

`Input`	start: officer, area, normal_max, age_limit. add_code: ID sesi dan teks ID batch, dengan atau tanpa ID:.
`Output`	start: integer ID sesi; add_code: tuple (batch_id, added), True untuk hasil baru dan False untuk scan ulang.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
from warehouse.sessions import Sessions
service = Sessions(db)
key = service.start('Faiz', 'Area A', 30, 90)
print(service.add_code(key, 'ID:DEMO-0001'))
print(service.add_code(key, 'DEMO-0001'))
print(len(service.results(key)))
```

Hasil yang diharapkan: Mencetak ('DEMO-0001', True), lalu ('DEMO-0001', False), lalu 1. checked_at dan snapshot pertama tetap sama.

Hubungan dengan fitur lain

session_ui meneruskan masukan ke add_code; hasil masuk check_results dan mark_check memperbarui aktivitas terakhir.

Kondisi error

- Petugas/area kosong, batas tidak valid, batch tidak dikenal, atau sesi sudah Selesai ditolak.

Catatan implementasi

- Contoh memerlukan db demo. Snapshot memuat master dan recommendation_at_check; edit master tidak mengubah results.

Referensi tes: test_sessions.py, test_recommendation_history.py

11. Menyimpan catatan dan menyelesaikan sesi yang terkunci

save_note memilih catatan sesi atau hasil berdasarkan batch_id. finish mensyaratkan sesi Draft yang mempunyai hasil. Status Selesai mengunci rekaman melalui pemeriksaan layanan dan trigger database.

Menentukan sasaran catatan dan memeriksa hasil UPDATE

Sumber: warehouse/sessions.py : 90-99 | Sessions.save_note

```
90  def save_note(self,key,note,batch_id=None):
91      note=str(note).strip()
92      if len(note)>2000:raise ValueError('Catatan maksimal 2000 karakter.')
93      with self.db:
94          self.require_draft(key)
95          if batch_id is None:
96              self.db.execute('UPDATE check_sessions SET note=? WHERE id=?',(note,key))
97          else:
98              result=self.db.execute('UPDATE check_results SET note=? WHERE session_id=? AND
98  > batch_id=?',(note,key,batch_id))
99              if result.rowcount!=1:raise ValueError('Barang belum ada di sesi ini.')
```

`note=str(note).strip()`	Masukan diubah menjadi teks lalu dibersihkan dari spasi di tepinya.
`if len(note)>2000:raise ValueError(...)`	Bentuk satu baris memuat pemeriksaan panjang dan exception yang menghentikan penyimpanan.
`with self.db: self.require_draft(key)`	Validasi status dan penulisan catatan berjalan dalam context transaksi SQLite.
`if batch_id is None:`	Perbandingan identitas None memilih catatan sesi; else memilih catatan barang.
`WHERE session_id=? AND batch_id=?`	Catatan hasil dibatasi oleh pasangan sesi dan batch, bukan batch saja.
`if result.rowcount!=1:`	Tidak adanya hasil untuk pasangan itu menghasilkan kesalahan, bukan sukses semu.

Memastikan hasil ada lalu mengunci status sesi

Sumber: warehouse/sessions.py : 101-107 | Sessions.finish

```
101 def finish(self,key):
102     with self.db:
103         self.require_draft(key)
104         if not self.db.execute('SELECT 1 FROM check_results WHERE
104 > session_id=?',(key,)).fetchone():
105             raise ValueError('Tambahkan setidaknya satu hasil sebelum menyelesaikan sesi.')
106         self.db.execute("UPDATE check_sessions SET status='Selesai',finished_at=? WHERE id=?",
107             (datetime.now().astimezone().isoformat(timespec='seconds'),key))
```

`def finish(self,key):`	Method menerima ID sesi; tidak mempunyai nilai return eksplisit.
`with self.db:`	Pemeriksaan dan pembaruan disahkan bersama ketika blok berhasil selesai.
`self.require_draft(key)`	Status diperiksa sebelum mencoba mengakhiri sesi atau menulis finished_at.

`SELECT 1 ... fetchone()`	Cukup mencari keberadaan satu hasil; sesi kosong menghasilkan None.
`status='Selesai',finished_at=?`	Satu UPDATE menyimpan status akhir dan waktu lokal yang memuat zona waktu.

Input dan hasil

`Input`	save_note: key, note, batch_id opsional. finish: key sesi yang berstatus Draft.
`Output`	Kedua method mengembalikan None; efeknya catatan tersimpan atau status Selesai dengan finished_at terisi.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
from warehouse.sessions import Sessions
service = Sessions(db)
key = service.start('Faiz', 'Area A')
service.add_code(key, 'DEMO-0001')
service.save_note(key, 'Cek label rak')
service.save_note(key, 'Kemasan perlu diperiksa', 'DEMO-0001')
service.finish(key)
print(service.get(key)['status'])
```

Hasil yang diharapkan: Mencetak Selesai. Catatan sesi dan barang tersimpan terpisah; pemanggilan save_note atau finish berikutnya memunculkan ValueError sesi terkunci.

Hubungan dengan fitur lain

session_ui menyimpan catatan melalui save_note; tindakan selesai memanggil finish sebelum menampilkan riwayat/laporan.

Kondisi error

- Catatan lebih dari 2000 karakter, batch belum ada dalam sesi, sesi kosong, atau sesi terkunci ditolak.

Catatan implementasi

- Contoh memerlukan db demo. Trigger locked_session/locked_result menolak edit SQL setelah selesai; tes mengharapkan IntegrityError.

Referensi tes: test_sessions.py

12. Mengeluarkan stok sekali saja dalam transaksi yang utuh

StockMovements.issue menyatukan pengurangan stok dan riwayat historis. operation_id mengembalikan transaksi lama pada percobaan ulang tanpa mengurangi stok lagi. Kunci penulisan sebelum pembacaan saldo mencegah dua koneksi menyetujui pengeluaran dari stok lama.

Kunci penulisan dan pemeriksaan transaksi yang sudah tercatat

Sumber: warehouse/stock_movements.py : 90-99 | StockMovements.issue

```
90  self.db.execute('BEGIN IMMEDIATE')
91  try:
92      prior = self.db.execute('SELECT * FROM stock_movements WHERE id=?',
93                              (movement_id,)).fetchone()
94      if prior is not None:
95          if any(prior[key] != value for key, value in payload.items()):
96              raise ValueError('ID transaksi sudah dipakai untuk data yang berbeda. Muat ulang
96  > formulir.')
97          result = self._record(prior)
98          self.db.commit()
99          return result
```

`self.db.execute('BEGIN IMMEDIATE')`	SQLite meminta kunci penulisan sebelum saldo dan riwayat transaksi dibaca.
`WHERE id=?`	movement_id hasil normalisasi menjadi identitas percobaan transaksi, bukan batch.
`if prior is not None:`	Cabang ini menangani percobaan ulang untuk ID transaksi yang telah ada.
`any(prior[key] != value for key, value in payload.items())`	Generator membandingkan setiap field payload; perubahan masukan membuat ID ditolak.
`self.db.commit(); return result`	Jika payload sama, transaksi ditutup dan record lama dikembalikan tanpa pengurangan ulang.

Mengurangi saldo, menyimpan snapshot, dan membatalkan bila gagal

Sumber: warehouse/stock_movements.py : 112-125 | StockMovements.issue

```
112     after = stock - quantity
113     record = dict(payload, id=movement_id, stock_before=stock, stock_after=after,
114                   recorded_at=datetime.now().astimezone().isoformat(timespec='microseconds'),
115                   snapshot=json.dumps(item, ensure_ascii=False))
116     self.db.execute('UPDATE batches SET stock=? WHERE id=?', (after, batch_id))
117     self.db.execute('''INSERT INTO stock_movements
118         (id,batch_id,occurred_on,recorded_at,quantity,stock_before,stock_after,officer,note,sn
118  > apshot)
119         VALUES (:id,:batch_id,:occurred_on,:recorded_at,:quantity,:stock_before,:stock_after,:
119  > officer,:note,:snapshot)''',
120         record)
121     self.db.commit()
122     return self._record(record)
123 except Exception:
124     self.db.rollback()
```

```
125         raise
```

`after = stock - quantity`	Saldo baru dihitung setelah jumlah keluar dipastikan tidak melebihi stok.
`record = dict(payload, id=..., stock_before=..., stock_after=...)`	Dictionary menggabungkan masukan bersih, saldo sebelum/sesudah, identitas, dan waktu pencatatan.
`json.dumps(item, ensure_ascii=False)`	Data master saat pengeluaran diserialisasi sebagai JSON; teks Unicode dipertahankan.
`UPDATE batches ...; INSERT INTO stock_movements ...`	Saldo dan riwayat ditulis melalui parameter SQL dalam transaksi sama.
`except Exception: self.db.rollback(); raise`	Kesalahan membatalkan semua perubahan lalu diteruskan kepada pemanggil.

Input dan hasil

`Input`	batch_id, quantity int/digit string 1–999999999, occurred_on YYYY-MM-DD, officer, note opsional, operation_id opsional.
`Output`	Dict riwayat: id, batch_id, quantity, stock_before, stock_after, occurred_on, recorded_at, officer, note, snapshot dan field tampilan.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
from datetime import date
from warehouse.stock_movements import StockMovements
service = StockMovements(db)
operation = 'OUT-' + 'A' * 32
first = service.issue('DEMO-0002', 2, date.today().isoformat(), 'Faiz',
> operation_id=operation)
again = service.issue('DEMO-0002', 2, date.today().isoformat(), 'Faiz',
> operation_id=operation)
print(first['stock_before'], first['stock_after'], first == again)
```

Hasil yang diharapkan: Pada db demo baru, mencetak 4 2 True. Stok akhir tetap 2 dan hanya satu riwayat tersimpan.

Hubungan dengan fitur lain

stock_ui mempertahankan ID operasi, memanggil issue, lalu membaca ulang inventory dan riwayat.

Kondisi error

- Jumlah/tanggal/petugas keliru, stok kurang, operation_id berbeda payload, atau transaksi aktif ditolak.

Catatan implementasi

- Contoh memerlukan db demo baru dan ID operasi belum dipakai. Tanggal keluar berada antara penerimaan dan hari ini; riwayat bertahan sesudah master diedit.

Referensi tes: test_stock_movements.py

Bagian C / Rekomendasi dan keputusan petugas

13. Membaca kebutuhan material dengan JOIN

recommendation_context() menyusun bukti kebutuhan yang tercatat untuk satu material. Site dibaca langsung; customer harus memiliki model unit yang kompatibel. Fungsi ini hanya membaca database dan menandai konteks sebagai Simulasi.

Query site, kecocokan model customer, dan hasil konteks

Sumber: warehouse/recommendation_data.py : 11-19 | recommendation_context

```
11  sites = [dict(row) for row in db.execute('''
12      SELECT site,demand_prev,demand_now,open_demand,site_stock
13      FROM site_demand WHERE material=? ORDER BY site''', (material,))]
14  customers = [dict(row) for row in db.execute('''
15      SELECT d.customer,d.unit_model,d.open_demand
16      FROM material_unit_compatibility c
17      JOIN customer_demand d ON d.unit_model=c.unit_model
18      WHERE c.material=? ORDER BY d.customer,d.unit_model''', (material,))]
19  return {'sites': sites, 'customers': customers, 'source': 'Simulasi'}
```

`[dict(row) for row in db.execute(...)]`	List comprehension mengubah setiap sqlite3.Row menjadi dictionary agar pembaca menggunakan nama kolom, lalu mengumpulkannya dalam list.
`'''SELECT ...'''`	Tiga tanda kutip membentuk string SQL beberapa baris; baris baru merupakan bagian string, bukan akhir perintah Python.
`WHERE material=? dan (material,)`	Tanda ? menerima parameter terikat. Koma membuat tuple satu elemen; nilai material tidak disisipkan langsung ke teks SQL.
`material_unit_compatibility c / customer_demand d`	c dan d adalah alias tabel. Awalan d.customer menyatakan kolom berasal dari tabel demand customer.
`JOIN ... ON d.unit_model=c.unit_model`	INNER JOIN hanya mengambil customer yang modelnya cocok dengan pemetaan material; model yang tidak cocok tidak menghasilkan baris.
`WHERE c.material=?`	Filter material tetap diperlukan setelah JOIN. Skema tanpa COLLATE NOCASE menggunakan kecocokan teks peka huruf besar-kecil; M-01 berbeda dari m-01.
`ORDER BY dan return {...}`	ORDER BY menetapkan urutan keluaran. Dictionary terakhir memisahkan sites, customers, serta label sumber; material tidak dikenal menghasilkan dua list kosong.

Input dan hasil

`Input`	Fixture: M-01 cocok dengan UNIT-TRUCK; Customer A meminta 4 unit dan customer lain memakai model berbeda.
`Output`	customers hanya memuat Customer A; source bernilai Simulasi.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
from warehouse.recommendation_data import recommendation_context
context = recommendation_context(db, 'M-01')
print([(c['customer'], c['open_demand']) for c in context['customers']])
print(recommendation_context(db, 'm-01')['customers'])
```

Hasil yang diharapkan: Dengan fixture di atas: [('A', 4)], lalu []. db merupakan koneksi fixture yang sudah disiapkan.

Hubungan dengan fitur lain

Sessions.add_code(), tampilan inventori, dan save_decision() memberikan material batch kepada fungsi ini sebelum memanggil recommend_for().

Kondisi error

- Tabel/kolom yang tidak tersedia menimbulkan sqlite3.Error; kebutuhan yang tidak cocok cukup menghasilkan list kosong.

Catatan implementasi

- Ini data dummy berbasis aturan, bukan AI terlatih. Tidak ada prediksi kebutuhan atau pembuatan tujuan saat membaca.

Referensi tes: tests/test_recommendation_data.py

14. Menyaring site dan menghitung jumlah usulan

_site_options() membentuk alternatif relokasi. Normal mensyaratkan demand naik. Untuk Melewati batas, kenaikan tidak diwajibkan; kandidat tetap memerlukan nama valid, integer nonnegatif, demand_now positif, dan open_demand melebihi site_stock. Jumlah usulan dibatasi stok batch dan diurutkan terbesar dahulu.

Validasi angka, ambang kenaikan, dan shortage

Sumber: warehouse/recommendations.py : 51-61 | _site_options

```
51  if (not isinstance(name, str) or not name.strip() or
52          any(type(value) is not int or value < 0 for value in
53              (prev, now, open_demand, site_stock)) or
54          now <= 0 or open_demand <= site_stock):
55      continue
56  if rising_only and (now - prev < 2 or now * 5 < prev * 6):
57      continue
58  shortage = open_demand - site_stock
59  units = min(stock, shortage)
60  target = name.strip()
61  intro = 'Jika fisik layak, ajukan redistribusi' if after_check else 'Pertimbangkan relokasi'
```

`any(... for value in (...))`	Generator memeriksa empat angka; any bernilai True segera setelah satu angka invalid. type(value) is not int juga menolak bool meskipun bool merupakan turunan int.
`continue`	Baris invalid dilewati dan perulangan maju ke site berikutnya; satu site buruk tidak menggagalkan seluruh hasil.
`rising_only and (... or ...)`	Saat rising_only=True, kandidat ditolak jika kenaikan kurang dari 2 unit atau 5×now kurang dari 6×prev. Kedua syarat kelulusan berarti minimal 2 unit dan 20%.
`shortage / min(stock, shortage)`	Shortage adalah open_demand dikurangi stok site. min memilih nilai lebih kecil agar usulan tidak melebihi kekurangan maupun stok batch.
`A if after_check else B`	Ekspresi kondisional memilih awalan tindakan. after_check=True menegaskan pemeriksaan kelayakan fisik sebelum redistribusi.

Membentuk dictionary opsi dan mengurutkannya

Sumber: warehouse/recommendations.py : 64-72 | _site_options

```
64      options.append(dict(kind='site', target=target,
65                          title=f'Relokasi ke Site {target}',
66                          reason=(f'{demand_note}; '
67                                  f'Site {target} kekurangan {shortage} unit.'),
68                          next_action=(f'{intro} {units} unit ke Site {target} '
69                                       'setelah konfirmasi petugas.'),
70                          suggested_units=units))
71  options.sort(key=lambda option: (-option['suggested_units'], option['target']))
72  return options
```

`options.append(dict(...))` dan `f'...{units}...'`	dict dengan argumen bernama membentuk satu opsi; append menambahnya ke list. f-string menyisipkan nilai ke alasan dan tindakan.

`sort(key=lambda option: (-..., ...))`	lambda menghasilkan tuple kunci: jumlah negatif membuat jumlah terbesar lebih dulu; kode tujuan memecahkan seri secara menaik. sort mengubah list yang sama.

Input dan hasil

`Input`	Stok batch 5; site A: demand 10→12, open_demand 9, site_stock 2; rising_only=True.
`Output`	Kenaikan tepat 2 unit/20% lolos; shortage=7 dan suggested_units=5.

Contoh penggunaan

Contoh dengan import lengkap.

```
from warehouse.recommendations import _site_options
site = dict(site='A', demand_prev=10, demand_now=12,
            open_demand=9, site_stock=2)
options = _site_options([site], 5, rising_only=True, after_check=False)
print([(o['target'], o['suggested_units']) for o in options])
```

Hasil yang diharapkan: [('A', 5)]

Hubungan dengan fitur lain

recommend_for() menggunakan rising_only=True untuk Normal dan False untuk Melewati batas; Warning tidak memanggil helper site.

Kondisi error

- Angka negatif, nama kosong, demand_now nol, atau kebutuhan yang sudah tertutup stok site dilewati.

Catatan implementasi

- Prev nol tetap wajib naik minimal 2 unit. Jumlah tiap opsi adalah alternatif, bukan alokasi gabungan atau perpindahan stok.

Referensi tes: test_recommendation_context.py

15. Customer dan urutan cabang rekomendasi

recommend_for() menjalankan aturan: validasi data, stok nol, Melewati batas, Warning, lalu Normal. Batas bawaan 30/90 hari. JOIN menyaring kecocokan material-model; helper customer memeriksa bentuk dan jumlah kebutuhan.

Customer valid dan jumlah penawaran

Sumber: warehouse/recommendations.py : 26-31 | _customer_options

```
26  if (not isinstance(name, str) or not name.strip() or
27          not isinstance(unit_model, str) or not unit_model.strip() or
28          type(open_demand) is not int or open_demand <= 0):
29      continue
30  units = min(stock, open_demand)
31  target = name.strip()
```

`not isinstance(..., str) or not ...strip()`	or menguji alasan penolakan berurutan. strip menghapus spasi tepi, sehingga nama atau model yang hanya berisi spasi dianggap kosong.
`type(open_demand) is not int`	Kebutuhan harus tepat integer positif; string angka, float, dan boolean ditolak melalui continue.
`min(stock, open_demand)`	Jumlah penawaran adalah kebutuhan atau stok batch, mana yang lebih kecil. Kecocokan material-model tidak diperiksa ulang oleh helper ini.

Cabang merah: customer lalu site

Sumber: warehouse/recommendations.py : 99-109 | recommend_for

```
99  if status == 'Melewati batas':
100     advice = dict(code='MELEWATI_BATAS', title='Periksa barang segera',
101                   reason=f'Umur {age} hari; {age-age_limit} hari melewati batas {age_limit}
101 > hari.',
102                   next_action=('Prioritaskan pemeriksaan fisik. Jika barang masih layak dan
102 > ada '
103                                'kebutuhan di site lain, pertimbangkan transfer.'), priority=3)
104     if context is None:
105         return advice
106     options = (_customer_options(context.get('customers', ()), stock) +
107                _site_options(context.get('sites', ()), stock,
108                              rising_only=False, after_check=True))
109     advice.update(source=context.get('source', 'Simulasi'), options=options)
```

`if status == 'Melewati batas'`	Cabang merah diperiksa sebelum Warning dan Normal; setiap cabang mengembalikan dictionary saran sehingga cabang berikutnya tidak dijalankan.
`if context is None: return advice`	None berarti konteks kebutuhan tidak diberikan. Fungsi langsung mengembalikan saran umur umum tanpa source atau options.
`context.get('customers', ())`	get memberi tuple kosong sebagai default ketika kunci tidak ada; helper tetap dapat melakukan perulangan tanpa kandidat.
`_customer_options(...) + _site_options(...)`	Operator + menggabungkan dua list. Ranking berlaku di masing-masing kelompok: seluruh customer mendahului seluruh site, bukan ranking jumlah gabungan.

`advice.update(...)`	update menambahkan source/options pada dictionary yang sama. Penggantian judul di cabang lain tidak mengubah code atau status umur.

Input dan hasil

`Input`	Tanggal masuk 2026-06-01, acuan 2026-10-05, stok 20; customer A butuh 3, site C kekurangan 14.
`Output`	MELEWATI_BATAS; customer A 3 unit lebih dulu, lalu site C 14 unit.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
from datetime import date
from warehouse.recommendations import recommend_for
context = {'customers': [dict(customer='A', unit_model='U1', open_demand=3)],
           'sites': [dict(site='C', demand_prev=10, demand_now=9,
                          open_demand=15, site_stock=1)]}
rec = recommend_for(dict(stock=20, received_date='2026-06-01'),
                    today=date(2026, 10, 5), context=context)
print(rec['code'], [(o['kind'], o['target'], o['suggested_units']) for o in rec['options']])
```

Hasil yang diharapkan: MELEWATI_BATAS [('customer', 'A', 3), ('site', 'C', 14)]

Hubungan dengan fitur lain

Normal memilih site dengan kenaikan demand; Warning memilih customer; merah memilih customer lalu site yang kekurangan meski demand menurun.

Kondisi error

- Tanggal/stok invalid menghasilkan DATA_PERLU_DIPERBAIKI; validasi ini mendahului STOK_HABIS.

Catatan implementasi

- Umur 24–30 menghasilkan NORMAL_DEKAT_WARNING. Cek fisik dan keputusan akhir tetap dilakukan petugas.

Referensi tes: test_recommendations.py, test_recommendation_context.py

16. Menyimpan keputusan secara atomik

save_decision() mencatat keputusan dalam sesi Draft. Kondisi Layak menjadi gerbang pemindahan/pemakaian. Inventori dan kebutuhan dibaca ulang agar tujuan yang berubah dapat ditolak sebelum commit.

Gerbang Layak, jenis tujuan, dan transaksi

Sumber: warehouse/recommendations.py : 211-223 | RecommendationService.save_decision

```
211 if choice in ('Perlu dipindahkan', 'Prioritaskan pemakaian',
212               'Pindahkan ke site', 'Tawarkan ke customer') and physical_condition != 'Layak':
213     raise ValueError('Barang harus dinyatakan Layak sebelum dipindahkan atau diprioritaskan.')
214 required_kind = TARGET_CHOICES.get(choice)
215 if required_kind:
216     if target_kind != required_kind or not isinstance(target_code, str) or not target_code:
217         raise ValueError('Pilih tujuan rekomendasi yang sesuai dengan tindakan.')
218 elif target_kind is not None or target_code is not None:
219     raise ValueError('Tujuan hanya dapat dipilih untuk relokasi atau penawaran.')
220 if self.db.in_transaction:
221     raise ValueError('Transaksi data lain masih berjalan. Selesaikan lalu coba lagi.')
222 
223 self.db.execute('BEGIN IMMEDIATE')
```

`choice in (...) and ... != 'Layak'`	in menguji keanggotaan empat pilihan; and mewajibkan kondisi Layak untuk pemindahan atau prioritas pemakaian. raise ValueError menghentikan simpan.
`TARGET_CHOICES.get(choice)`	Dictionary memetakan pilihan ke site/customer. Tujuan wajib memiliki jenis yang sesuai; pilihan umum harus tanpa target_kind dan target_code.
`in_transaction / BEGIN IMMEDIATE`	Transaksi aktif ditolak. BEGIN IMMEDIATE meminta kunci penulis sejak awal, sehingga pemeriksaan dan penyimpanan berada dalam satu transaksi.

Hitung ulang, cocokkan tujuan, dan snapshot

Sumber: warehouse/recommendations.py : 258-269 | RecommendationService.save_decision

```
258 advice = recommend_for(item, normal_max=session['normal_max'],
259                        age_limit=session['age_limit'], today=now.date(),
260                        context=context)
261 if required_kind and not any(
262         option['kind'] == target_kind and option['target'] == target_code
263         for option in advice.get('options', ())):
264     raise ValueError('Tujuan tidak tersedia dalam rekomendasi barang saat ini.')
265 if choice == 'Perlu dipindahkan' and status != 'Melewati batas':
266     raise ValueError('Pilihan Perlu dipindahkan hanya untuk barang Melewati batas.')
267 values = (physical_condition,choice,target_kind,target_code,reason,session['officer'],
268           now.isoformat(timespec='seconds'),age,status,
269           json.dumps(advice,ensure_ascii=False))
```

`today=now.date(), normal_max=session[...]`	Argumen bernama membuat hitung ulang memakai tanggal lokal saat simpan serta batas umur sesi, bukan tanggal saat dialog dibuka.
`not any(... for option in ...)`	Generator mencari pasangan kind/target yang cocok persis dalam opsi terkini. Target yang hilang menyebabkan ValueError dan rollback.
`values = (...) dan session['officer']`	Tuple menyiapkan parameter SQL berurutan; petugas diambil dari sesi. Umur/status keputusan disimpan terpisah dari hasil scan.

`json.dumps(advice, ensure_ascii=False)`	Dictionary saran diserialisasi menjadi JSON snapshot; karakter non-ASCII tetap terbaca. UPDATE merevisi keputusan Draft yang sama; INSERT membuat keputusan baru.

Input dan hasil

`Input`	Fixture: sesi 7 Draft, BAT-X tercatat, stok 5, site A masih valid; kondisi Layak dan alasan terisi.
`Output`	Keputusan menuju A berstatus Belum selesai; stok tetap 5.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
from warehouse.recommendations import RecommendationService
decision = RecommendationService(db).save_decision(
    7, 'BAT-X', 'Layak', 'Pindahkan ke site',
    'Kebutuhan Site A sudah dikonfirmasi.',
    target_kind='site', target_code='A')
print(decision['target_code'], decision['action_status'])
```

Hasil yang diharapkan: A Belum selesai, dengan fixture di atas; tidak terjadi perubahan lokasi atau stok.

Hubungan dengan fitur lain

ID manual atau QR mencatat hasil pengecekan. Pemeriksaan lolos → commit; exception → rollback tanpa keputusan parsial.

Kondisi error

- Sesi terkunci, stok habis, belum ada hasil pengecekan, atau tindakan lama belum selesai dapat menolak simpan.

Catatan implementasi

- Tujuan tetap valid tetapi jumlah berubah memakai snapshot baru; tidak ada pemeriksaan versi dialog atau masa berlaku demand.

Referensi tes: test_recommendations.py, test_recommendation_context.py

17. Penyelesaian terpisah dan proyeksi histori

complete() menambah satu rekaman penyelesaian tanpa mengubah snapshot keputusan. report_data() merangkai fakta saat pengecekan dengan field keputusan tersimpan. Waktu scan, keputusan, dan completion merupakan tiga kejadian berbeda yang tidak boleh disamakan.

Completion satu kali dan commit/rollback

Sumber: warehouse/recommendations.py : 303-312 | RecommendationService.complete

```
303         if action['completed_at'] is not None:
304             raise ValueError('Tindakan ini sudah diselesaikan.')
305         self.db.execute('''INSERT INTO recommendation_completions
306             (decision_id,officer,completed_at,note) VALUES(?,?,?,?)''',
307             (decision_id,officer,datetime.now().astimezone().isoformat(timespec='seconds'),note))
308         self.db.commit()
309         return self._decision('d.id=?', (decision_id,))
310     except Exception:
311         self.db.rollback()
312         raise
```

`if action['completed_at'] is not None`	Penyelesaian kedua ditolak. Identitas keputusan tetap merujuk keputusan awal; completion ditulis ke tabel recommendation_completions yang terpisah.
`VALUES(?,?,?,?)`	Empat placeholder mengikat ID keputusan, petugas penyelesaian, waktu, dan catatan. Petugas penyelesaian dapat berbeda dari petugas sesi.
`astimezone().isoformat(timespec='seconds')`	Waktu lokal disimpan sebagai teks ISO beserta offset zona waktu, dengan ketelitian detik.
`commit / except Exception / rollback / raise`	Commit mengesahkan insert; jika operasi gagal, rollback membatalkan transaksi lalu raise meneruskan exception kepada pemanggil.

Snapshot saat cek dan field keputusan untuk laporan

Sumber: warehouse/reports.py : 49-60 | report_data

```
49      immutable_fields=('id','physical_condition','choice','reason','officer','decided_at',
50                        'age_at_decision','aging_status','recommendation_code','recommendation_title
50    > ',
51                        'recommendation_reason','recommendation_next_action','recommendation_priorit
51    > y',
52                        'target_kind','target_code')
53      for row in rows:
54          checked_day=date.fromisoformat(row['checked_at'][:10])
55          row['recommendation']=row.get('recommendation_at_check') or recommend_for(
56              row,normal_max=session['normal_max'],age_limit=session['age_limit'],today=checked_day)
57          recorded=decisions.for_result(key,row['id'])
58          row['decision']={field:recorded[field] for field in immutable_fields} if recorded else
58    > None
59      return dict(session=session,rows=rows,counts={s:sum(r['status']==s for r in rows) for s in
59    > STATUSES},
60                  generated_at=datetime.now().astimezone().isoformat(timespec='seconds'))
```

`row['checked_at'][:10]`	Slice mengambil YYYY-MM-DD dari timestamp scan; date.fromisoformat mengubahnya menjadi tanggal untuk fallback data lama.

`row.get(...) or recommend_for(...)`	or memakai rekomendasi scan jika tersedia. Jika tidak, fungsi menghitung dari snapshot inventori pada tanggal scan, tanpa membaca demand terkini.
`{field:recorded[field] for field in immutable_fields}`	Dictionary comprehension memproyeksikan hanya field keputusan yang disebutkan. Completion dan action_status tidak masuk keluaran laporan.
`sum(r['status']==s for r in rows)`	Perbandingan menghasilkan boolean; sum menghitung True sebagai 1 untuk total setiap status umur saat pengecekan.

Input dan hasil

`Input`	Fixture: sesi 7 selesai, satu keputusan tindakan belum memiliki completion.
`Output`	action_status menjadi Selesai; isi rows laporan tetap sama setelah completion.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
from warehouse.recommendations import RecommendationService
from warehouse.reports import report_data
before = report_data(db, 7)['rows']
done = RecommendationService(db).complete(
    decision_id, 'NIK-02', 'Relokasi telah ditangani.')
after = report_data(db, 7)['rows']
print(done['action_status'], before == after)
```

Hasil yang diharapkan: Selesai True, dengan fixture tersebut; stok dan lokasi tetap seperti sebelum pencatatan completion.

Hubungan dengan fitur lain

Detail Draft menghitung saran terkini; detail Selesai memilih snapshot keputusan, atau saran scan bila belum diputuskan. Laporan memakai saran scan.

Kondisi error

- Keputusan Tidak perlu tindakan, ID invalid, catatan kosong, dan completion berulang ditolak.

Catatan implementasi

- Saran scan dapat berbeda dari snapshot keputusan. Pencatatan selesai tidak mengeksekusi transfer; perubahan inventori dicatat melalui alur terpisah.

Referensi tes: test_recommendation_reports.py, test_session_recommendation_display.py, test_recommendations.py

Bagian D / UI, kamera, laporan, dan executable

18. Navigasi Tkinter dan pemeriksaan pergantian hari

Application.show membangun halaman yang dipilih, sedangkan check_day memperbarui jam dan data ketika tanggal berubah. Keduanya berjalan sebagai callback pada utas antarmuka Tkinter. Perhatikan perbedaan pemanggilan langsung dan callback terjadwal.

Memilih halaman dan mengganti isi wadah

Sumber: warehouse/ui.py : 187-200 | Application.show

```
187  def show(self, page):
188      self.stop_camera()
189      self.page=page
190      self.title.configure(text=page+'  /  Data simulasi')
191      self.footer.configure(text='Pemantauan sementara  •  Tidak membuat laporan' if
191  > page=='Live'
192                            else 'Data simulasi  •  Tersimpan di perangkat ini')
193      for name,b in self.nav.items():
194          selected=name==page or (page=='Lokasi' and name=='Data Barang')
195          b.configure(bg='#164F84' if selected else SIDEBAR)
196          self.nav_markers[name].configure(bg='#69C3FF' if selected else SIDEBAR)
197      for child in self.body.winfo_children():child.destroy()
198      self.rows=self.inventory.list_batches()
199      if page=="Dashboard":self.dashboard()
200      elif page=="Data Barang":self.inventory_page()
```

`self.stop_camera()`	Tanda kurung menjalankan metode segera; yang ditutup ialah jendela kamera pengecekan yang disimpan pada camera_window.
`self.page=page`	Atribut objek menyimpan nama halaman untuk callback berikutnya, termasuk pemeriksaan pergantian hari.
`for name,b in self.nav.items()`	items() menghasilkan pasangan kunci dan tombol; unpacking membagikan keduanya ke name dan b.
`selected=name==page or (...)`	Ekspresi Boolean juga menyorot Data Barang ketika halaman Lokasi aktif; ekspresi if/else memilih warna.
`for child in self.body.winfo_children():child.destroy()`	Daftar widget anak diambil lalu dihancurkan sebelum data dibaca ulang dan halaman berikutnya dibuat.

Callback berulang dengan after

Sumber: warehouse/ui.py : 537-542 | Application.check_day

```
537  def check_day(self):
538      self.header_time.configure(text=datetime.now().strftime('%d %b %Y  %H:%M'))
539      if date.today()!=self.today:
540          self.today=date.today()
541          if self.page not in ("Lokasi","Pengecekan"):self.show(self.page)
542      self.timer=self.root.after(30000,self.check_day)
```

`date.today()!=self.today`	Operator != mendeteksi pergantian tanggal lokal komputer, bukan perubahan setiap menit.
`self.page not in ("Lokasi","Pengecekan")`	Uji keanggotaan mengecualikan dua halaman tersebut dari pembangunan ulang saat hari berganti.
`self.root.after(30000,self.check_day)`	Referensi metode tanpa () dijadwalkan sekitar 30 detik lagi; nilai kembali disimpan sebagai ID timer.

Input dan hasil

`Input`	Nama halaman seperti Dashboard; objek app dengan root, navigasi, inventori, dan wadah Tk yang sudah dibuat.
`Output`	Judul dan penanda halaman berubah; isi body diganti. Callback harian menjadwalkan pemanggilan berikutnya.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
# Konteks: app sudah memiliki root Tk dan database.
app.show('Dashboard')
print(app.page)
```

Hasil yang diharapkan: Dashboard; widget dashboard terbentuk pada body.

Hubungan dengan fitur lain

Tombol navigasi memasang lambda p=page:self.show(p); argumen default menangkap nama tiap tombol. close membatalkan timer sebelum root dihancurkan.

Kondisi error

- Memanggil show sesudah root dihancurkan dapat memicu TclError; pastikan callback dibatalkan saat penutupan.

Catatan implementasi

- show tidak menghentikan LiveMonitor. Timer Live terpisah memanggil poll_live sekitar setiap 120 ms, sehingga pemantauan hidup ketika berpindah halaman.

Referensi tes: test_foundation_ui.py, test_live_ui.py

19. Agregasi section dan pembukaan tampak samping rak

Dashboard menggabungkan lokasi dengan kolom dan baris sama menjadi satu section pada tampak atas. Klik section memanggil DashboardPage.pick_section untuk membuka tampak samping kolom. Tingkat menjadi sel terpisah.

Pengelompokan berdasarkan pasangan koordinat

Sumber: warehouse/dashboard.py : 18-29 | aggregate_sections

```
18  def aggregate_sections(cells):
19      """Project top-down sections from the three independent storage levels."""
20      sections = {}
21      for cell in cells:
22          key = (cell['column'], cell['row'])
23          section = sections.setdefault(key, dict(column=key[0], row=key[1], cells=[], rows=[]))
24          section['cells'].append(cell)
25          section['rows'].extend(cell['rows'])
26      for section in sections.values():
27          section['cells'].sort(key=lambda cell: cell['level'])
28          section.update(aggregate(section['rows']))
29      return sections
```

`key = (cell['column'], cell['row'])`	Tuple yang dapat dipakai sebagai kunci dictionary mengelompokkan seluruh tingkat pada koordinat kolom/baris sama.
`sections.setdefault(key, dict(...))`	setdefault mengembalikan section yang ada atau memasukkan dictionary awal ketika kuncinya belum tersedia.
`section['cells'].append(cell); section['rows'].extend(cell['rows'])`	append memasukkan satu sel; extend menambahkan isi rows tanpa membuat daftar bersarang.
`sort(key=lambda cell: cell['level'])`	lambda memberi kunci urut tingkat; sort mengubah cells sebelum aggregate memperbarui ringkasan.

Callback klik menuju jendela kolom

Sumber: warehouse/dashboard_ui.py : 349-364 | DashboardPage.pick_section

```
349 def pick_section(self, key, column, row):
350     """Show one side elevation, using the clicked section to select its column."""
351     self._hide_rack_tip()
352     self.highlight = None
353     self.selected_rack = key
354     self.selected_column = column
355     self.selected_row = row
356     self.map_canvas.set_data(self.data['racks'], self.highlight, key)
357     rack = self.data['racks'][key]
358     screen_w = self.app.root.winfo_screenwidth()
359     screen_h = self.app.root.winfo_screenheight()
360     win = getattr(self, 'rack_window', None)
361     if win is None or not win.winfo_exists():
362         win = self.dialog(f'Rak {key[1]} · Kolom {column:02d}',
363                           f'{min(940, screen_w - 80)}x{min(670, screen_h - 80)}')
364     else:
```

`self.selected_rack = key`	Atribut pilihan menyimpan kunci rak; atribut column dan row berikutnya menyimpan section yang diklik.
`self.map_canvas.set_data(...)`	Pemanggilan metode memperbarui penyorotan peta menggunakan data rak yang sudah dimuat pada dashboard.
`getattr(self, 'rack_window', None)`	getattr menyediakan None bila atribut jendela belum pernah dibuat, sehingga pembukaan pertama tetap valid.
`if win is None or not win.winfo_exists()`	or memeriksa objek kosong atau widget tertutup; min membatasi ukuran dialog terhadap layar.

Input dan hasil

`Input`	cells berisi column, row, level, dan rows; klik menyertakan key rak seperti ('A','B01'), kolom, serta baris.
`Output`	Dictionary sections berkunci tuple; dialog tampak samping menampilkan kolom terpilih beserta pilihan baris.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
# Konteks: dashboard sudah memuat rak A/B01.
dashboard.pick_section(('A', 'B01'), 3, 2)
print(dashboard.selected_column, dashboard.selected_row)
```

Hasil yang diharapkan: 3 2; tampak samping Kolom 03 terbuka dan Baris 02 disorot.

Hubungan dengan fitur lain

overview memanggil aggregate_sections dan aggregate_columns. RackMap meneruskan klik ke pick_section; tombol sebelumnya/berikutnya memanggil callback yang sama.

Kondisi error

- Kunci rak tidak tersedia memicu KeyError; gunakan key dari self.data['racks'].

Catatan implementasi

- Bagian lanjutan fungsi memakai sorted dan set untuk kolom/baris, serta reverse=True agar Tingkat 03 tampil di atas. Batch stok nol tidak menentukan status agregat.

Referensi tes: test_dashboard.py, test_dashboard_ui.py

20. Decode QR, antrean frame, dan pelepasan perangkat kamera

decode_frame mengembalikan teks dan sudut QR. CameraWorker membaca kamera pada utas terpisah lalu mengirim paket melalui antrean; SQLite tetap di utas antarmuka.

Multi-QR dengan fallback satu QR

Sumber: warehouse/camera.py : 31-41 | decode_frame

```
31  def decode_frame(frame, detector=None):
32      detector = detector or cv2.QRCodeDetector()
33      height,width=frame.shape[:2]
34      scale=min(1,1280/max(width,height))
35      source=cv2.resize(frame,(round(width*scale),round(height*scale))) if scale<1 else frame
36      ok,codes,points,_=detector.detectAndDecodeMulti(source)
37      found=[(code,pts/scale) for code,pts in zip(codes,points) if code] if ok and points is not
37  > None else []
38      if not found:
39          code,points,_=detector.detectAndDecode(source)
40          if code and points is not None:found=[(code,points.reshape(4,2)/scale)]
41      return found
```

`height,width=frame.shape[:2]`	Slice mengambil dua dimensi gambar; unpacking menempatkan tinggi dan lebar pada variabel berbeda.
`scale=min(1,1280/max(width,height))`	Rasio maksimal satu; gambar besar diperkecil dan koordinat dibagi scale agar kembali ke gambar asli.
`zip(codes,points) if code`	zip memasangkan teks dan sudut; comprehension membuang teks kosong. Nama _ menandai hasil decode yang tidak dipakai.
`points.reshape(4,2)/scale`	Fallback mengubah sudut satu QR menjadi empat pasangan koordinat dengan bentuk yang seragam.

Frame terbaru dan cleanup ketika utas berhenti

Sumber: warehouse/camera.py : 84-99 | CameraWorker.run

```
84      while not self.stop_event.is_set():
85          found=decode_frame(frame,detector)
86          packet=(frame,found)
87          try:self.frames.get_nowait()
88          except queue.Empty:pass
89          self.frames.put_nowait(packet)
90          if self.stop_event.wait(.06):break
91          ok,frame=cap.read()
92          if not ok or frame is None:raise ValueError('Sambungan kamera terputus. Periksa
92  > perangkat, lalu aktifkan kamera kembali.')
93  except Exception as exc:
94      self.messages.put(('error',str(exc)))
95  finally:
96      if cap is not None:cap.release()
97      self.stop_event.set()
98      DEVICE_LOCK.release()
99      self.messages.put(('stopped',None))
```

`while not self.stop_event.is_set()`	Event adalah sinyal berhenti lintas utas; loop berlanjut selama sinyal belum disetel.
`get_nowait(); except queue.Empty:pass`	Pengambilan tanpa menunggu membuang paket lama bila ada; put_nowait kemudian mengisi antrean dengan frame terbaru.
`self.stop_event.wait(.06)`	Menunggu paling lama 0,06 detik sambil menerima sinyal berhenti; True menyebabkan break tanpa pembacaan berikutnya.
`finally: ... DEVICE_LOCK.release()`	Cleanup melepas capture dan lock, lalu mengirim stopped, baik setelah berhenti normal maupun exception.

Input dan hasil

`Input`	frame berupa array gambar BGR; CameraWorker.start menerima indeks perangkat kamera.
`Output`	Daftar (code, points), paket (frame, found), serta pesan error/stopped pada antrean.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
# Konteks: frame memuat label DEMO-0001.
found = decode_frame(frame)
print(found[0][0], found[0][1].shape)
```

Hasil yang diharapkan: ID:DEMO-0001 (4, 2).

Hubungan dengan fitur lain

start memperoleh DEVICE_LOCK tanpa menunggu dan membuat utas daemon; frames berkapasitas satu. UI mengambil paket pada callback poll.

Kondisi error

- Kamera terpakai atau terputus menghasilkan pesan kesalahan; pilih perangkat lain atau aktifkan kembali kamera.

Catatan implementasi

- open_device meminta 1280×720 dan 15 FPS; perangkat/backend menentukan hasil aktual. Kamera pengecekan menolak beberapa QR berbeda dan menyimpan hasil hanya sesudah petugas menekan konfirmasi.

Referensi tes: test_camera_reports.py, test_live_camera_handoff.py

21. LiveMonitor: konfirmasi dua poll dan masa tahan QR

LiveMonitor menyimpan pengamatan QR sementara di memori. Kode tampil setelah dua pembacaan terproses dan ditahan dua detik sejak terakhir terlihat. Modul ini tidak menulis database ataupun membuat hasil pengecekan.

Dedup dalam frame dan pembaruan track

Sumber: warehouse/live_monitor.py : 126-139 | LiveMonitor._observe

```
126  seen = set()
127  for code, points in found:
128      if not code or code in seen:
129          continue
130      seen.add(code)
131      corners = tuple((float(x), float(y)) for x, y in points)
132      track = self._tracks.get(code)
133      if track is None:
134          self._tracks[code] = _Track(code, corners, now, now)
135      else:
136          track.points = corners
137          track.last_seen = now
138          track.reads += 1
139          track.visible = True
```

`seen = set()`	Set baru untuk setiap pengamatan menyimpan kode yang sudah dihitung pada frame ini.
`if not code or code in seen: continue`	Kode kosong dan label identik berikutnya dilewati; dua label sama pada satu frame bukan dua konfirmasi.
`tuple((float(x), float(y)) for x, y in points)`	Generator mengubah tiap sudut menjadi pasangan float, lalu tuple menyimpan koordinat dengan bentuk tetap.
`track = self._tracks.get(code)`	get mengembalikan None jika kode baru; dictionary menyimpan track menurut kode dan mempertahankan urutan masuknya.
`track.last_seen = now; track.reads += 1`	Track lama memperbarui waktu, menambah penghitung dengan operator +=, dan ditandai visible untuk frame terbaru.

Menghapus pengamatan yang melewati TTL

Sumber: warehouse/live_monitor.py : 141-144 | LiveMonitor._expire

```
141  def _expire(self, now):
142      for code, track in tuple(self._tracks.items()):
143          if now - track.last_seen >= self.hold_seconds:
144              del self._tracks[code]
```

`tuple(self._tracks.items())`	Salinan pasangan kode/track memungkinkan dictionary dihapus selama iterasi tanpa mengubah iterable yang sedang dibaca.
`now - track.last_seen >= self.hold_seconds`	Operator >= memasukkan batas tepat dua detik; del membuang track sehingga kemunculan berikutnya memerlukan konfirmasi ulang.

Input dan hasil

Input	Paket frame dari worker, daftar kode/sudut, dan waktu monoton dari clock.
Output	LiveSnapshot berisi frame, detections terkonfirmasi, messages, serta running; track hilang setelah TTL.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
# Konteks: monitor aktif dan worker palsu seperti tes.
worker.frame('f1', [('ID:A', points)])
print(len(monitor.poll().detections))
worker.frame('f2', [('ID:A', points)])
print(len(monitor.poll().detections))
```

Hasil yang diharapkan: 0 lalu 1; dua poll masing-masing memproses satu frame mengonfirmasi kode A.

Hubungan dengan fitur lain

Application.poll_live menjalankan poll sekitar setiap 120 ms. poll membuang frame lama, menghapus track kedaluwarsa, mengamati frame terakhir, lalu membuat snapshot.

Kondisi error

- confirm_reads < 1 atau hold_seconds negatif memicu ValueError.

Catatan implementasi

- Dua frame antrean dalam satu poll hanya menghasilkan satu pengamatan. Navigasi halaman mempertahankan monitor; penghentian selesai ketika pesan stopped diproses.

Referensi tes: test_live_monitor.py, test_live_ui.py, test_live_camera_handoff.py

22. Membaca laporan dari snapshot sesi

report_data membaca hasil konfirmasi pengecekan. Material, stok, umur, dan rekomendasi mengikuti snapshot saat cek. Keputusan dibaca terpisah dari recommendation_decisions berdasarkan sesi dan batch; keputusan Draft dapat direvisi sampai terkunci. Perubahan inventori tidak mengganti fakta pemeriksaan.

Sesi, hasil, dan daftar field historis

Sumber: warehouse/reports.py : 44-52 | report_data

```
44  def report_data(db,key):
45      service=Sessions(db)
46      session=service.get(key)
47      rows=service.results(key)
48      decisions=RecommendationService(db)
49      immutable_fields=('id','physical_condition','choice','reason','officer','decided_at',
50                        'age_at_decision','aging_status','recommendation_code','recommendation_t
50  > itle',
51                        'recommendation_reason','recommendation_next_action','recommendation_pri
51  > ority',
52                        'target_kind','target_code')
```

`service=Sessions(db)`	Layanan menerima koneksi tersedia; get mengambil sesi dan results membaca snapshot JSON hasil.
`immutable_fields=(...)`	Tuple mendefinisikan field keputusan yang masuk laporan; status penyelesaian tindakan tidak dimasukkan ke daftar ini.

Melengkapi snapshot dan menghitung ringkasan

Sumber: warehouse/reports.py : 53-60 | report_data

```
53  for row in rows:
54      checked_day=date.fromisoformat(row['checked_at'][:10])
55      row['recommendation']=row.get('recommendation_at_check') or recommend_for(
56          row,normal_max=session['normal_max'],age_limit=session['age_limit'],today=checked_day)
57      recorded=decisions.for_result(key,row['id'])
58      row['decision']={field:recorded[field] for field in immutable_fields} if recorded else
58  > None
59  return dict(session=session,rows=rows,counts={s:sum(r['status']==s for r in rows) for s in
59  > STATUSES},
60              generated_at=datetime.now().astimezone().isoformat(timespec='seconds'))
```

`row['checked_at'][:10]`	Slice mengambil tanggal YYYY-MM-DD dari timestamp; fromisoformat mengubahnya menjadi objek date untuk perhitungan lama.
`row.get('recommendation_at_check') or recommend_for(...)`	Operator or memilih rekomendasi tersimpan yang tersedia; fallback untuk hasil lama memakai tanggal cek dan batas umur sesi.
`decisions.for_result(key,row['id'])`	Pasangan ID sesi dan batch memilih keputusan yang terkait hasil itu, bukan keputusan terbaru dari sesi lain.
`{field:recorded[field] for field in immutable_fields} if recorded else None`	Dictionary comprehension menyalin field terpilih; conditional expression menghasilkan None bila keputusan belum dicatat.

`sum(r['status']==s for r in rows)`	Generator menghasilkan Boolean; sum menghitung True sebagai satu untuk setiap kategori status pada dictionary counts.

Input dan hasil

`Input`	db berupa koneksi aplikasi dan key berupa ID sesi yang ada.
`Output`	Dictionary session, rows, counts, dan generated_at; setiap row dilengkapi recommendation dan decision.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
# Konteks: session_id adalah sesi selesai berisi satu hasil.
data = report_data(db, session_id)
print(data['session']['status'], len(data['rows']))
```

Hasil yang diharapkan: Selesai 1; data barang mengikuti snapshot saat cek meskipun inventori sudah berubah.

Hubungan dengan fitur lain

ReportPage dan fungsi ekspor memakai report_data. Sessions.results memuat snapshot kemudian menempelkan checked_at, age, status, dan note yang tersimpan.

Kondisi error

- ID sesi tidak dikenal memicu ValueError 'Sesi tidak ditemukan.'; timestamp rusak dapat gagal pada fromisoformat.

Catatan implementasi

- generated_at adalah waktu penyusunan data laporan, dengan offset zona waktu. Ekspor memakai nilai ini tanpa menghitung ulang. Laporan Draft dapat dibuat dan ditandai belum selesai.

Referensi tes: test_camera_reports.py, test_recommendation_reports.py

23. Ekspor PDF: file sementara, font, dan teks aman

ReportLab menyusun PDF pada file sementara. atomic_file mengganti file tujuan setelah penulisan berhasil. Helper Paragraph memastikan tanda markup pada catatan tetap tampil sebagai teks.

Context manager untuk penyimpanan atomik

Sumber: warehouse/reports.py : 63-71 | atomic_file

```
63  @contextmanager
64  def atomic_file(path):
65      path=Path(path)
66      with tempfile.NamedTemporaryFile(dir=path.parent,suffix=path.suffix,delete=False) as
66  > file:temporary=Path(file.name)
67      try:
68          yield temporary
69          os.replace(temporary,path)
70      finally:
71          if temporary.exists():temporary.unlink()
```

`Path(path)`	Konstruktor menormalkan masukan string menjadi objek path yang menyediakan parent, suffix, exists, dan unlink.
`NamedTemporaryFile(...,delete=False)`	File sementara ditutup setelah with; delete=False mempertahankannya pada direktori tujuan untuk penulis PDF.
`yield temporary`	Dekorator @contextmanager di atas fungsi memberi path ini kepada blok with milik pemanggil, lalu melanjutkan sesudah blok selesai.
`os.replace(temporary,path); finally`	replace dijalankan hanya bila penulisan sukses; finally membersihkan file sementara tersisa termasuk ketika penulis mengalami exception.

Pilihan font dan escaping Paragraph

Sumber: warehouse/reports.py : 141-152 | export_pdf

```
141 regular,bold='Helvetica','Helvetica-Bold'
142 font_root=Path(os.environ.get('WINDIR','C:/Windows'))/'Fonts'
143 if (font_root/'arial.ttf').exists():
144     if 'WarehouseArial' not in pdfmetrics.getRegisteredFontNames():
145         pdfmetrics.registerFont(TTFont('WarehouseArial',str(font_root/'arial.ttf')))
146         pdfmetrics.registerFont(TTFont('WarehouseArialBold',str(font_root/'arialbd.ttf')))
147     regular,bold='WarehouseArial','WarehouseArialBold'
148 normal=ParagraphStyle('body',fontName=regular,fontSize=9,leading=13,spaceAfter=6,wordWrap='LTR
148 > ')
149 small=ParagraphStyle('cell',parent=normal,fontSize=8,leading=11,spaceAfter=0)
150 header=ParagraphStyle('head',parent=small,fontName=bold,textColor=colors.white)
151 title=ParagraphStyle('title',parent=normal,fontName=bold,fontSize=20,leading=25,textColor=colo
151 > rs.HexColor('#11283F'))
152 def p(value,style=normal):return Paragraph(escape(str(value)).replace('\n','<br/>'),style)
```

`os.environ.get('WINDIR','C:/Windows')`	get membaca direktori Windows dengan nilai pengganti; operator / pada Path membangun lokasi folder Fonts.
`'WarehouseArial' not in pdfmetrics.getRegisteredFontNames()`	Uji keanggotaan mencegah registrasi berulang; bila Arial tersedia, TTFont mendaftarkan font normal dan tebal.

`ParagraphStyle(..., parent=normal)`	parent mewarisi gaya dasar; small mengubah ukuran teks, header memilih font tebal.
`escape(str(value)).replace('\n',' ')`	str mengubah nilai menjadi teks, escape melindungi <, >, dan &, lalu newline diberi markup pergantian baris yang terkontrol.

Input dan hasil

`Input`	data dari report_data dan path PDF pada folder tujuan yang tersedia.
`Output`	PDF A4 landscape dengan tabel, catatan lengkap, footer nomor halaman, serta label Draft jika relevan.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
# Konteks: data laporan sudah dimuat dan folder tersedia.
export_pdf(data, Path('laporan') / 'CHK-000001.pdf')
```

Hasil yang diharapkan: File CHK-000001.pdf dibuat; catatan '<periksa & kemasan>' tampil literal.

Hubungan dengan fitur lain

export_pdf menyusun story dan LongTable(repeatRows=1); doc.build memakai atomic_file. Callback footer berlaku pada semua halaman.

Kondisi error

- Folder hilang, izin ditolak, atau file terkunci dapat memicu OSError.

Catatan implementasi

- Helvetica menjadi fallback ketika Arial tidak tersedia. Ekspor tidak menyimpan ulang hasil pengecekan ke database.

Referensi tes: test_camera_reports.py, test_recommendation_reports.py

24. Entry point, build Windows, dan path aplikasi portable

warehouse_app.main memilih verifikasi paket atau membuka antarmuka. scripts.build_windows.main membuat paket baru dengan validasi keluaran. Jalur database frozen mengikuti executable portable; sumber memakai folder proyek.

Argumen build dan penolakan folder yang berisiko tertimpa

Sumber: scripts/build_windows.py : 108-121 | main

```
108  def main():
109      parser = argparse.ArgumentParser()
110      parser.add_argument('--output', default='Aplikasi_Warehouse_Check')
111      args = parser.parse_args()
112      if sys.platform != 'win32':
113          raise SystemExit('Build ini harus dijalankan pada Windows 64-bit.')
114      output = (ROOT / args.output).resolve()
115      if not output.is_relative_to(ROOT) or output == ROOT:
116          raise SystemExit('Folder hasil harus berada di dalam folder proyek.')
117      if output.exists():
118          raise SystemExit('Folder hasil sudah ada. Gunakan --output nama_baru agar database
118 > pengguna tidak tertimpa.')
119      seed = ROOT / 'data_uji' / 'warehouse_demo_seed.db'
120      for source in (seed, ROOT / 'warehouse' / 'package_check.py'):
121          if not source.is_file():
```

`parser.add_argument('--output', default=...)`	Argparse mendaftarkan pilihan nama folder keluaran; parse_args membaca argumen baris perintah.
`sys.platform != 'win32'`	Pemeriksaan platform menolak build selain Windows; string win32 juga dipakai pada Windows 64 bit.
`output = (ROOT / args.output).resolve()`	Operator / menyusun path; resolve memperoleh lokasi absolut sebelum validasi batas folder proyek.
`if output.exists(): raise SystemExit(...)`	Folder yang sudah ada menghentikan program sebelum packaging; pengguna perlu memilih nama keluaran baru.

Pemilihan mode dan event loop desktop

Sumber: warehouse_app.py : 8-21 | main

```
8   def main():
9       if len(sys.argv) == 3 and sys.argv[1] == '--verify-package':
10          from warehouse.package_check import verify_package
11          return verify_package(sys.argv[2])
12      root = tk.Tk()
13      try:
14          Application(root)
15      except Exception as exc:
16          root.withdraw()
17          messagebox.showerror("Warehouse Check", f"Aplikasi gagal dibuka:\n{exc}", parent=root)
18          root.destroy()
19          return 1
20      root.mainloop()
```

```
21      return 0
```

`len(sys.argv) == 3 and sys.argv[1] == '--verify-package'`	Short-circuit and memeriksa jumlah argumen terlebih dahulu; mode ini meneruskan path laporan JSON ke verify_package.
`root = tk.Tk(); Application(root)`	Tk membuat jendela utama dan Application mengisi aplikasi; konstruksi dilakukan sebelum event loop berjalan.
`except Exception as exc`	Kesalahan konstruksi Application ditangkap, jendela disembunyikan, pesan ditampilkan, lalu root dihancurkan dan kode satu dikembalikan.
`root.mainloop(); return 0`	Event loop melayani tombol dan timer sampai jendela ditutup; keberhasilan normal mengembalikan kode nol.

Input dan hasil

`Input`	Argumen --output atau --verify-package beserta path laporan; resource, seed, dan dependensi tersedia.
`Output`	Antarmuka, laporan verifikasi JSON, atau folder portable baru menurut mode yang dipilih.

Contoh penggunaan

Contoh pemanggilan; konteks objek/variabel mengikuti penjelasan bagian ini.

```
1  # Konteks CLI; ini ilustrasi, bukan dijalankan di tutorial.
2  sys.argv = ['warehouse_app.py', '--verify-package', 'cek_paket.json']
3  exit_code = main()
```

Hasil yang diharapkan: Pemeriksaan lulus menghasilkan exit_code 0 dan cek_paket.json dengan ok=true.

Hubungan dengan fitur lain

Build menjalankan PyInstaller, membuat database bersih, menguji executable dari cwd lain, lalu menyalin hasil ke output.

Kondisi error

- Kegagalan tk.Tk terjadi sebelum blok try; folder output lama dan lokasi di luar proyek ditolak dengan SystemExit.

Catatan implementasi

- application_root memakai folder executable saat frozen; sumber memakai resource_root. Database: data/warehouse_demo.db; resource frozen: _internal.

Referensi tes: test_package_build.py, tests/test_runtime_paths.py

Menjalankan, menguji, dan melanjutkan proyek

Jalankan dari root proyek yang berisi warehouse_app.py. Pada komputer proyek, interpreter yang digunakan rilis berada pada .build/venv/Scripts/python.exe. Pada komputer baru, buat venv Windows x64 yang menyertakan Tcl/Tk dan pasang dependency dari packaging/requirements-windows.lock.txt.

```
$projectPython = ".\.build\venv\Scripts\python.exe"
& $projectPython .\warehouse_app.py
& $projectPython -m unittest discover -s . -p "test_*.py"
& $projectPython -m unittest discover -s tests -p "test_*.py"
```

Jika Tk tidak menemukan init.tcl pada lingkungan proyek ini, variabel proses TCL_LIBRARY dan TK_LIBRARY dapat menunjuk runtime _tcl_data/_tk_data di dalam _internal paket. Gunakan versi Tcl/Tk yang sesuai interpreter. Tes UI memerlukan sesi desktop; kamera/printer fisik tidak dibuktikan oleh QR sintetis. Rilis 4 Oktober 2026 memiliki bukti 129 + 19 = 148 tes lulus di docs/exe_update_20261004.json.

Tiga jenis data

`Source`	data/warehouse_demo.db; dapat berisi riwayat pengguna/pengembangan.
`Seed`	data_uji/warehouse_demo_seed.db; dataset untuk pembuatan paket, bukan database yang dipakai runtime paket.
`Executable`	Aplikasi_Warehouse_Check_Rekomendasi_Final/data/warehouse_demo.db; terpisah dari source.

Alur yang harus dipertahankan

Input form atau Excel -> validasi Inventory -> SQLite -> QR ID batch -> sesi Draft lewat ID manual/QR -> snapshot hasil -> pemeriksaan fisik -> keputusan/tujuan -> sesi Selesai -> laporan. Live hanya menampilkan pembacaan QR sementara. Pencatatan tindak lanjut selesai tidak memindahkan stok secara otomatis.

Urutan perubahan yang aman

Ubah service aturan terlebih dahulu, hubungkan UI pada service tersebut, tambahkan tes regresi dengan DB sementara, lalu build paket baru. Perubahan schema menggunakan migrasi berikutnya, bukan reset database pengguna. Saat update pengguna, uji pada salinan DB dan pertahankan backup paket/DB yang berpasangan.

Indeks modul dan batas implementasi

`domain.py / database.py`	Aturan umur/alamat, koneksi, migrasi, backup, seed.
`inventory.py / locations.py`	Master batch dan lokasi.
`labels.py / inventory_excel.py`	QR/label dan impor/ekspor inventory.
`sessions.py / stock_movements.py`	Snapshot cek, catatan, penguncian, audit pengeluaran.
`recommendations.py / recommendation_data.py`	Saran, konteks Site/Customer, keputusan, completion.
`ui.py / *_ui.py / inventory_table.py`	Shell dan komponen desktop.
`dashboard.py / rack_map.py`	Agregasi lokasi, peta dan target klik.
`camera.py / live_monitor.py`	Capture/decode dan track sementara.
`reports.py / report_ui.py`	Laporan historis dan preview PDF.
`runtime_paths.py / scripts/build_windows.py`	Path portable serta packaging.

Batas versi ini

Belum ada login/role, integrasi SAP/API, konfigurasi RTSP atau banyak kamera simultan, prediksi demand berbasis histori, maupun ledger semua perubahan stok. Peta berasal dari alamat master, bukan sensor lokasi. Keputusan petugas masih melalui hasil sesi; hasil itu dapat dibuat dengan ID manual tanpa kamera.

Dokumen pendamping dalam proyek

ARCHITECTURE.md dan docs/PANDUAN_PENGEMBANG.md berisi peta serta prosedur operasional pengembangan. DATA_DAN_ATURAN_BISNIS.md, SISTEM_REKOMENDASI.md, dan ANTARMUKA_DAN_KAMERA.md memperinci kontrak bisnis. PDF ini berfokus pada pembacaan kode dan sintaks; semua rujukan source ditulis relatif terhadap root proyek.

Angka baris dan cuplikan mengikuti kode saat penyusunan. Setelah source diubah, regenerasikan PDF agar contoh tetap sesuai implementasi.